

인공지능과 피지컬 컴퓨팅(블루투스 통신)

가린 편집본 — 기관 계정 폴더 경로·개인 파일 이름·블루투스 기기 주소·학급 게시물 화면·외부 홍보 이미지 속 인물을 회색 상자로 지웠어요. 나머지는 원본 교안 그대로예요.



목차

01 Media pipe 모듈을 이용한 Hand Tracking

02 Thonny 세팅

03 손가락 인식

04 마이크로 파이썬을 이용한 피지컬 컴퓨팅

05 블루투스 통신

Media Pipe 모듈을 이용한 Hand Tracking

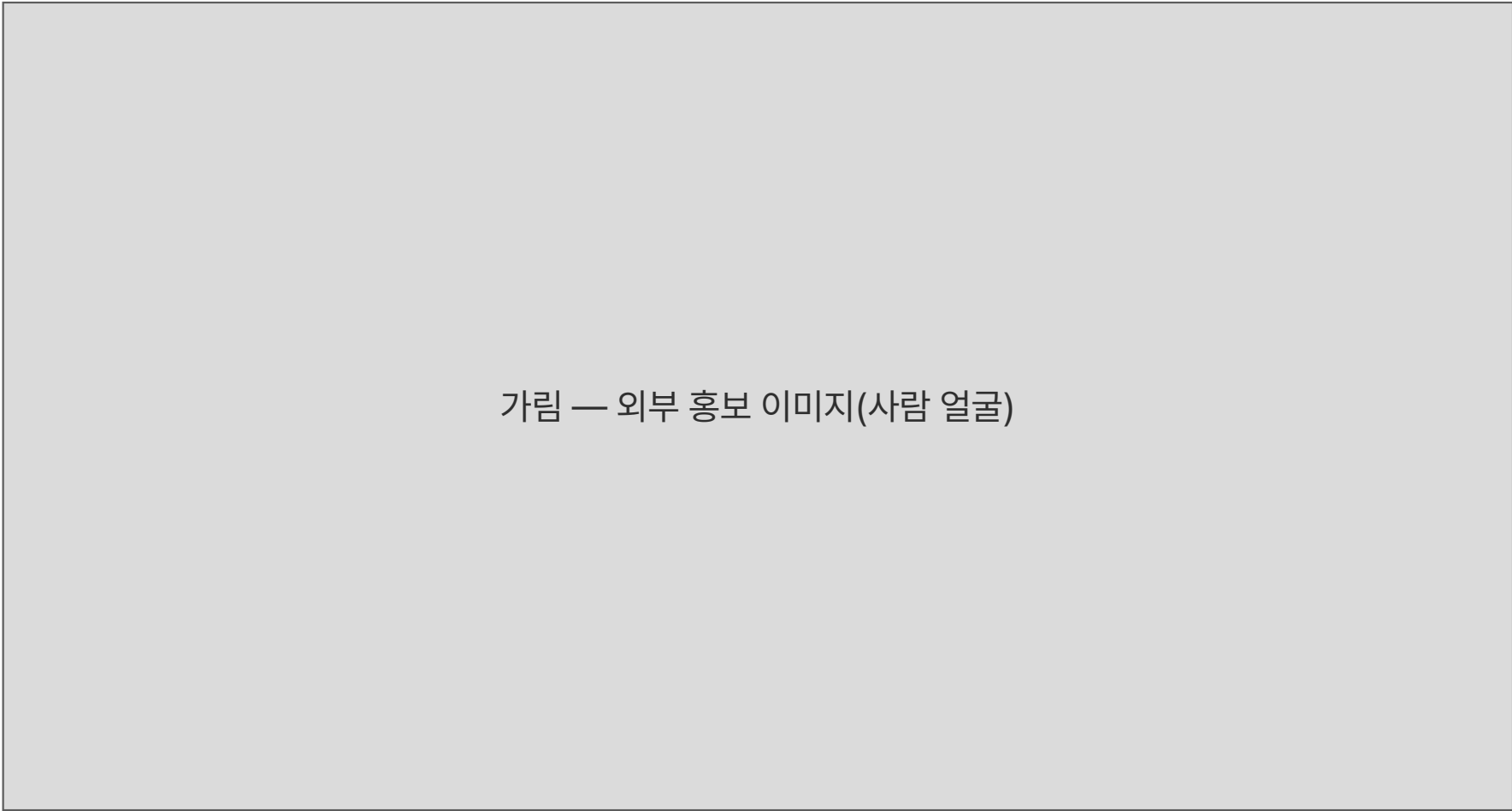
Open CV?

- OpenCV : 오픈소스 Computer Vision 라이브러리
- <https://opencv.org/>



MediaPipe?

- Mediapipe : 구글 오픈소스 Machine Learning 라이브러리
- <https://developers.google.com/mediapipe>



가림 — 외부 홍보 이미지(사람 얼굴)

05 OpenCV 모듈 설치(도구-패지키관리)

Thonny -

가림 — 기관 계정 폴더 경로

파일 편집 보기 실행 도구 도움말



파일 ×

이 컴퓨터

C: \ Users \ user

- + Contacts
- + Downloads
- + Favorites
- + Links
- + mlink
- + mlink1
- + Music

(p168

C:\Program Files (x86)\Thonny\python.exe를 위한 패키지 관리

opencv|

<인스톨>

absl-py
adafruit-board-toolkit
astroid
asttokens
async-timeout

검색 결과

[opencv-python](#)

Wrapper package for OpenCV python bindings.

05 OpenCV 동작 확인(다음 코드 입력)

```
import cv2

cap=cv2.VideoCapture(0)

while True:
    ret, frame=cap.read()
    if not ret : break
    cv2.imshow('Hand Tracking',frame)

    if cv2.waitKey(1)==27:break
cap.release()
cv2.destroyAllWindows()
```

open_window.py

파일에 왼쪽 파란색깔 코드 입력

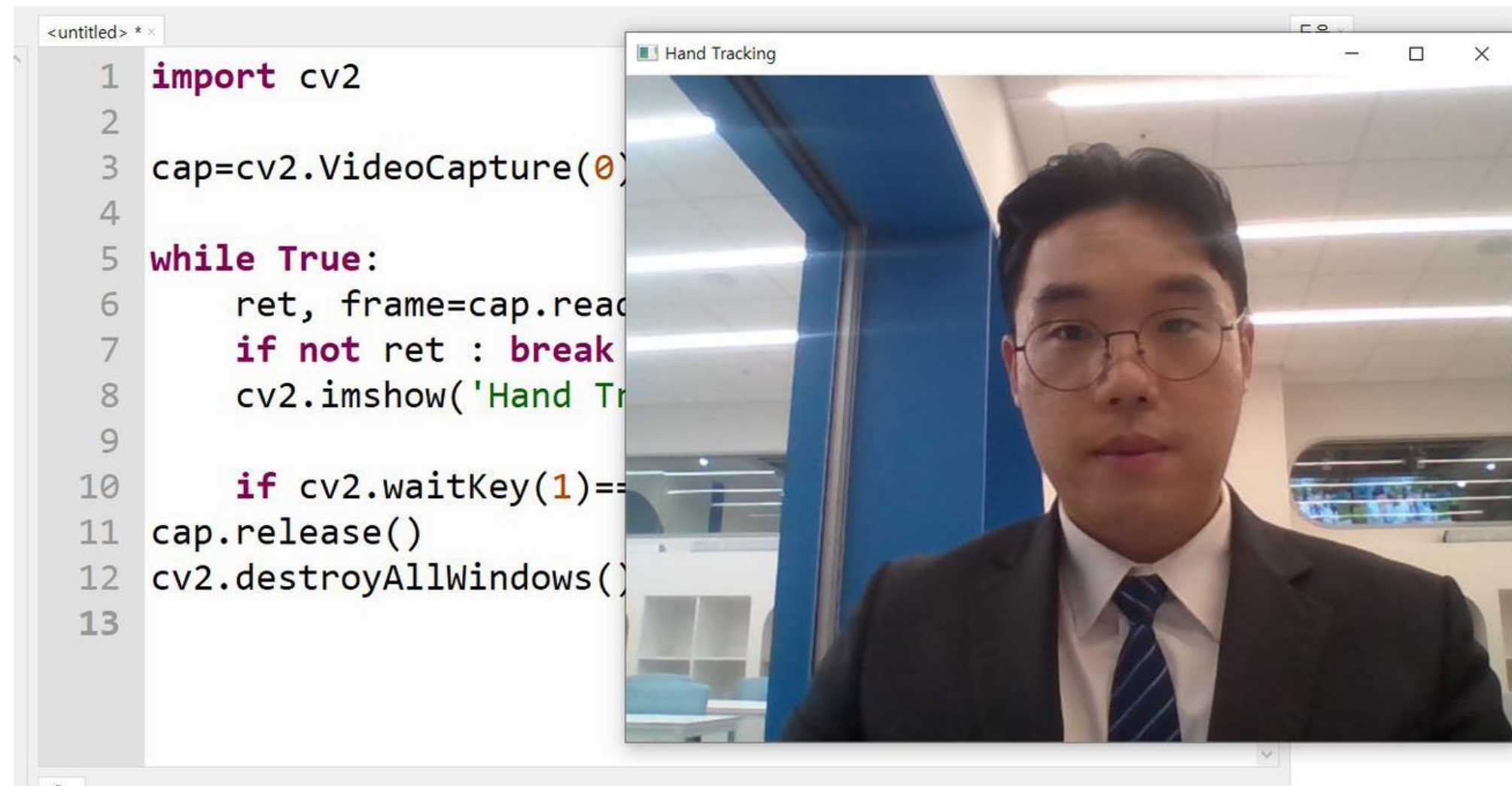
05 OpenCV 동작 확인(다음 코드 실행)



좌측 상단에

초록색깔 재생 버튼 클릭

05 OpenCV 동작 확인(코드 실행 화면)



다음과 같은
화면이 나오게 되면 세팅 완료.
정상 작동 가능

OpenCV로 캠 화면을 띄우기

```
1 import cv2
2 # 캠 불러오기
3 cap=cv2.VideoCapture(0)
4 #카메라가 동작하는 동안 계속하여 반복
5 while True:
6     # 비디오 스트림(카메라) 캡처
7     ret, frame=cap.read()
8     if not ret : break #읽어오는데 실패했다면 종료하기
9     #현재 프레임 이미지를 화면에 띄우기
10    cv2.imshow('Hand Tracking',frame)
11    # 'ESC'키를 누르면 종료시키기
12    if cv2.waitKey(1)==27:break
13 #종료 됐을 때 자원 해제(카메라 프로세스 종료)
14 cap.release()
15 cv2.destroyAllWindows()
```

- 따라하기
- 대소문자 지키기
- 들여쓰기 동일하게
- 줄 바꿈 동일하게
- 파이썬은 들여쓰기와 줄바꿈으로 문장을 인식

화면이 안 켜질 경우
fn+f1(녹음방지모드, 카메라해킹)

화면에 사람 표시

OpenCV로 캠 화면 사이즈 변경/가져오기

```
2  # 캠 불러오기
3  cap=cv2.VideoCapture(0)
4  cap.set(cv2.CAP_PROP_FRAME_WIDTH,1080)
5  width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
6  height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```

4. 캠 화면의 가로 길이를 1080으로 변경

5-6 캠 화면의 가로와 세로 길이를 가져오기

캠 화면이 변경이 안되는 경우

1. 카메라 지원 : 사용중인 카메라가 해당 해상도를 지원 x
(해상도 : 화면에 표시되는 픽셀의 수. 1920X1080은 너비 1920픽셀, 높이 1080픽셀)

2. 코드 재작성

```
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1080)
```

```
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
```

3. 카메라 드라이버 문제

1) 드라이버 업데이트

2) 드라이버 재설치

05 mediapipe 모듈 설치(도구-패지키관리)

C:\Thonny\python.exe를 위한 패키지 관리

mediapipe

esptool
flatbuffers
fonttools
intelhex
isort
jax
jaxlib
jedi
kiwisolver
matplotlib
mccabe
mediapipe
ml-dtypes
mouseinfo
mypy
mypy-extensions
numpy
opencv-contrib-python
opencv-python
opt-einsum
packaging
paramiko
parso

mediapipe

설치 버전: 0.10.21

설치: <C:\Users\User\AppData\Roaming\Python\Python310\site-packages>

마지막 안정화 버전: 0.10.32

요약: MediaPipe is the simplest way for researchers and ML solutions and applications for mobile, edge, cloud and the web.

제작자: The MediaPipe Authors

License: Apache 2.0

홈페이지: <https://github.com/google/mediapipe>

PyPI 페이지: <https://pypi.org/project/mediapipe/>

요구사항: absl-py~=2.3, numpy, sounddevice~=0.5, flatbuffers~=25.9, opencv-contrib-python, matplotlib

업그레이드

설치 제거

...

닫기

C:\Thonny\python.exe를 위한 패키지 관리

mediapipe

esptool
flatbuffers
fonttools
intelhex
isort
jax
jaxlib
jedi
kiwisolver
matplotlib
mccabe
mediapipe
ml-dtypes
mouseinfo
mypy
mypy-extensions
numpy
opencv-contrib-python
opencv-python
opt-einsum
packaging
paramiko
parso

mediapipe

설치 버전: 0.10.21

설치: <C:\Users\User\AppData\Roaming\Python\Python310\site-packages>

마지막 안정화 버전: 0.10.32

요약: MediaPipe is the simplest way for researchers and ML solutions and applications for mobile, edge, cloud and the web.

제작자: The MediaPipe Authors

License: Apache 2.0

홈페이지: <https://github.com/google/mediapipe>

PyPI 페이지: <https://pypi.org/project/mediapipe/>

요구사항: absl-py~=2.3, numpy, sounddevice~=0.5, flatbuffers~=25.9, opencv-contrib-python, matplotlib

업그레이드

설치 제거

...

닫기

추가적인 설치 / 업...

필요 버전

0.10.21

요구사항:

- * absl-py
- * attrs>=19.1.0
- * flatbuffers>=2.0
- * jax
- * jaxlib
- * matplotlib
- * numpy<2
- * opencv-contrib-python
- * protobuf<5,>=4.25.3
- * sounddevice>=0.4.4
- * sentencepiece

☐ 의존성 업그레이드

설치

취소

Mediapipe의 Hands 모델 사용하기

```
1 import cv2
2 import mediapipe as mp
3 mp_hands=mp.solutions.hands
4 mp_drawing=mp.solutions.drawing_utils
5 hands=mp_hands.Hands(max_num_hands=1)
```

1. Opencv 라이브러리를 불러옴.
2. Mediapipe 라이브러리를 불러옴
3. 손 감지 기능을 mp_hands 변수에 할당
4. 손의 점을 그리는 기능을 mp_drwaing에 할당
5. Hands는 mp_hands 내에 정의된 클래스. Hands클래스의 인스턴스를 생성하고, 이를 'hands'변수에 할당. max_num_hands=1은 인스턴스 생성할 때 전달하는 파라미터. 한 번에 감지할 수 있는 손의 최대개수를 1개로 설정.

Mediapipe의 Hands 모델 사용하기

```
13 ~ while True:
14     # 비디오 스트림(카메라) 캡처
15     ret, frame=cap.read()
16     if not ret : break #읽어오는데 실패했다면 종료하기
17     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
18     results=hands.process(image)
19 ~     if results.multi_hand_landmarks:
20 ~         for hand_landmarks in results.multi_hand_landmarks:
21 ~             mp_drawing.draw_landmarks(
22                 frame,
23                 hand_landmarks,
24                 mp_hands.HAND_CONNECTIONS)
```

17. cv2 BGR형태의 그림을 RGB로 변경 후 image에 저장.

18. 캡처된 프레임에서 손 인식 수행

19. 손 인식 결과가 있는지 확인

- 카메라로 찍힌 화면에서 인식된 손들의 정보를 리스트 형태로 제공

20-21 : 인식된 손들을 하나씩 확인하면서 손의 뼈대를 프레임 위에 그리기

- 리스트 안에 있는 각각의 손의 정보('hand_landmarks')를 차례대로 가져와서 처리

자유 자재로 꾸미기

```
1 import cv2
2 import mediapipe as mp
3 mp_hands=mp.solutions.hands
4 mp_drawing=mp.solutions.drawing_utils
5 hands=mp_hands.Hands(max_num_hands=1)
6
7 line_style = mp_drawing.DrawingSpec(
8     color = (166,151,18),
9     thickness = 3
10 )
11 circle_style = mp_drawing.DrawingSpec(
12     color = (166,114,31),
13     thickness = 2,
14     circle_radius = 3
```

7-14 : 선분 스타일과 원 스타일 설정
- 색깔은 BGR 순서로 작성

- 색깔 팔레트 찾기

<https://color.adobe.com/ko/explore>

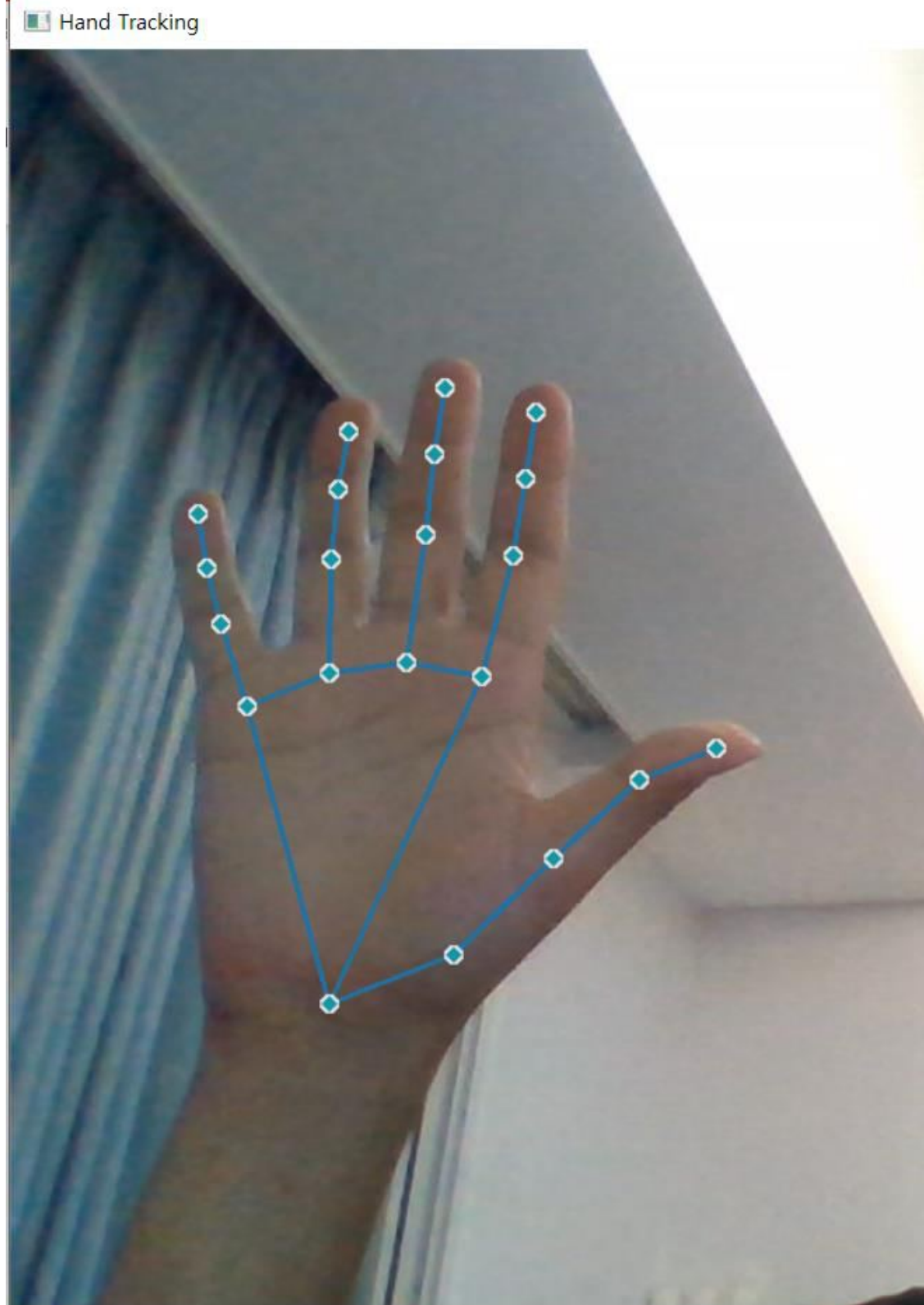
손 뼈대, 랜드마크 점 추가하기

```
23 √ while True:
24     # 비디오 스트림(카메라) 캡처
25     ret, frame=cap.read()
26     if not ret : break #읽어오는데 실패했다면 종료하기
27     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
28     results=hands.process(image)
29 √     if results.multi_hand_landmarks:
30 √         for hand_landmarks in results.multi_hand_landmarks:
31 √             mp_drawing.draw_landmarks(
32                 frame,
33                 hand_landmarks,
34                 mp_hands.HAND_CONNECTIONS,
35                 line_style,
36                 circle_style)
```

35-36 : 선분 스타일, 원 스타일 두 개의 매개변수를 추가한 뼈대 그리기

Circle_style, line_style이 맞음. 학생들한테 함수에 들어가는 순서도 중요하다는 부분 강조하기 위해서 일부러 line_style, circle_style바꿈

중간점검-실행



어떤 필요성이 느껴지는가?

좌우 반전하기

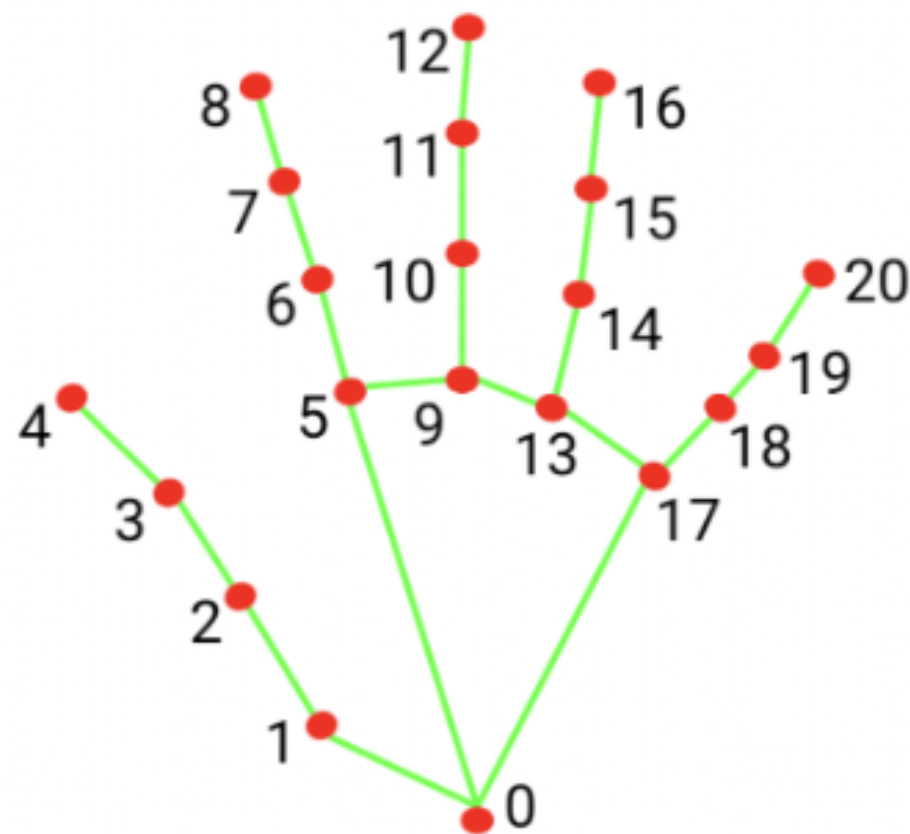
```
23 while True:
24     # 비디오 스트림(카메라) 캡처
25     ret, frame=cap.read()
26     frame=cv2.flip(frame,1)
```

화면을 좌우 반전

실행결과 확인

Mediapipe Hands 모듈의 이해

손가락 번호



0. WRIST	11. MIDDLE_FINGER_DIP
1. THUMB_CMC	12. MIDDLE_FINGER_TIP
2. THUMB_MCP	13. RING_FINGER_MCP
3. THUMB_IP	14. RING_FINGER_PIP
4. THUMB_TIP	15. RING_FINGER_DIP
5. INDEX_FINGER_MCP	16. RING_FINGER_TIP
6. INDEX_FINGER_PIP	17. PINKY_MCP
7. INDEX_FINGER_DIP	18. PINKY_PIP
8. INDEX_FINGER_TIP	19. PINKY_DIP
9. MIDDLE_FINGER_MCP	20. PINKY_TIP
10. MIDDLE_FINGER_PIP	

- 손가락 랜드마크의 번호
- 검지 손가락의 좌표를 가져오려면?
- 손가락이 접혔다는 것을 체크하려면?

손가락 좌표 확인하기

```
1 finger_point = hand_landmarks.landmark[8]
2 print(finger_point)
```

- 적절한 곳에 넣어보자.
- 출력되는 값을 확인하자.

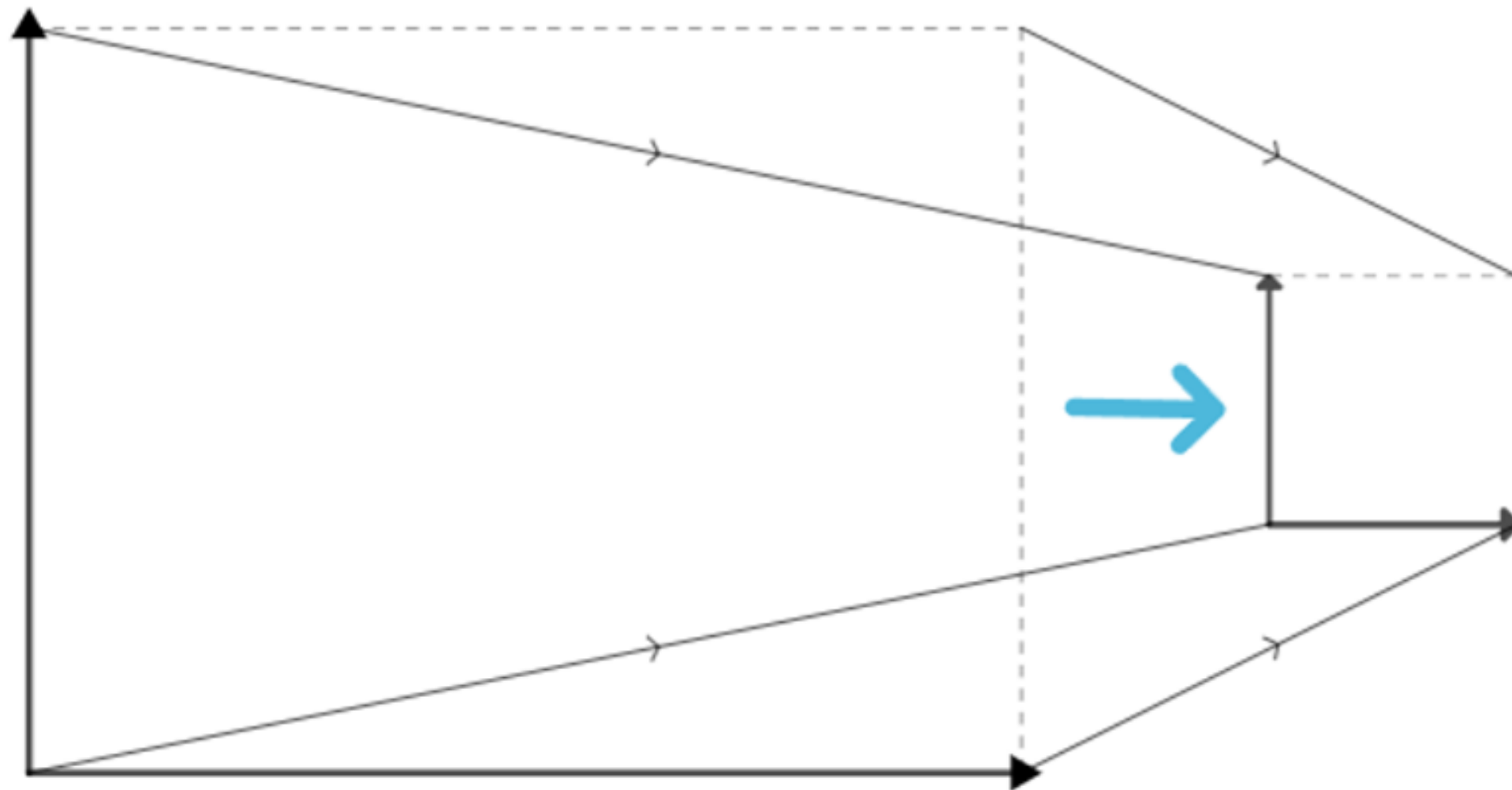
손가락 좌표 확인하기

```
23 while True:
24     # 비디오 스트림(카메라) 캡처
25     ret, frame=cap.read()
26     frame=cv2.flip(frame,1)
27
28     if not ret : break #읽어오는데 실패했다면 종료하기
29     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
30     results=hands.process(image)
31     if results.multi_hand_landmarks:
32         for hand_landmarks in results.multi_hand_landmarks:
33             finger_point=hand_landmarks.landmark[8]
34             print(finger_point)
```

x : 화면의 가로 방향을 기준으로 한 위치(왼쪽 끝이 0.0, 오른쪽 끝이 1.0)-이미지 벗어나면 음수
y : 화면의 세로 방향을 기준으로 한 위치(위쪽 끝이 0.0, 아래쪽 끝이 1.0)-이미지 벗어나면 음수
z : 손이 카메라로부터 얼마나 떨어져 있는지 나타내는 깊이(값이 작을수록 카메라에 가깝고, 클수록 카메라에 멀다.)

좌표는 정규화된 값.

표준화(Normalize)된 좌표



- 860, 640을 1,1으로 축소
- 다시 되돌리려면?

화면상의 손가락 위치 정확히 출력하기

```
20 width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
21 height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
22 #카메라가 동작하는 동안 계속하여 반복
23 while True:
24     # 비디오 스트림(카메라) 캡처
25     ret, frame=cap.read()
26     frame=cv2.flip(frame,1)
27
28     if not ret : break #읽어오는데 실패했다면 종료하기
29     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
30     results=hands.process(image)
31     if results.multi_hand_landmarks:
32         for hand_landmarks in results.multi_hand_landmarks:
33             finger_point=hand_landmarks.landmark[8]
34             finger_x=int(finger_point.x*width)
35             finger_y=int(finger_point.y*height)
36             print(finger_x, finger_y)
```

- 20-21 : 캠 화면의 너비와 높이
- 34-35 : 감지한 손가락의 위치계산
→ 화면 상의 위치로 복원
→ OpenCV에서 좌표를 지정 할때는 반드시 정수로(픽셀)

손가락 랜드마크에 점 그리기(검지 손가락 인식)

```
32  ✓ for hand_landmarks in results.multi_hand_landmarks:
33      finger_point=hand_landmarks.landmark[8]
34      finger_x=int(finger_point.x*width)
35      finger_y=int(finger_point.y*height)
36  ✓ mp_drawing.draw_landmarks(
37          frame,
38          hand_landmarks,
39          mp_hands.HAND_CONNECTIONS,
40          line_style,
41          circle_style)
42      #def circle(img, center, radius, color, thickness, lineType)
43      cv2.circle(frame, (finger_x,finger_y),30,(255,255,255),10)
```

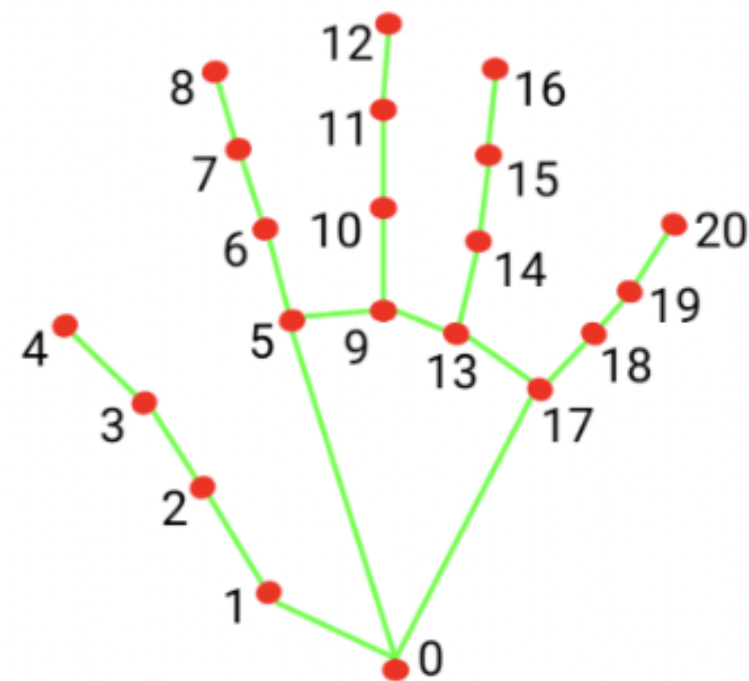
- 점을 찍는 함수가 없기 때문에 원을 이용
- 42 : circle함수의 정의를 확인하면서 순서대로 필요한 값을 넣기
- 43 : 값넣은 상황. radius, color 값을 변경하면서 실제로 반영되는지 확인

접혀있는 손가락 감지하여 출력하기

```
23 compare = [(2,4,17),(5,8,0),(9,12,0),(13,16,0),(17,20,0)]
24 #카메라가 동작하는 동안 계속하여 반복
25 while True:
26     # 비디오 스트림(카메라) 캡처
27     ret, frame=cap.read()
28     frame=cv2.flip(frame,1)
```

- 길이를 비교하여 손가락이 접혀 있는지 판별
- 비교할 대상을 튜플형태로

손가락 번호



- | | |
|-----------------------|-----------------------|
| 0. WRIST | 11. MIDDLE_FINGER_DIP |
| 1. THUMB_CMC | 12. MIDDLE_FINGER_TIP |
| 2. THUMB_MCP | 13. RING_FINGER_MCP |
| 3. THUMB_IP | 14. RING_FINGER_PIP |
| 4. THUMB_TIP | 15. RING_FINGER_DIP |
| 5. INDEX_FINGER_MCP | 16. RING_FINGER_TIP |
| 6. INDEX_FINGER_PIP | 17. PINKY_MCP |
| 7. INDEX_FINGER_DIP | 18. PINKY_PIP |
| 8. INDEX_FINGER_TIP | 19. PINKY_DIP |
| 9. MIDDLE_FINGER_MCP | 20. PINKY_TIP |
| 10. MIDDLE_FINGER_PIP | |

접혀 있는 손가락 감지하여 출력하기

```
25 while True:
26     # 비디오 스트림(카메라) 캡처
27     ret, frame=cap.read()
28     frame=cv2.flip(frame,1)
29
30     if not ret : break #읽어오는데 실패했다면 종료하기
31     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
32     results=hands.process(image)
33     if results.multi_hand_landmarks:
34         for hand_landmarks in results.multi_hand_landmarks:
35             h1=hand_landmarks.landmark
36             folding=""
```

- 35 : 자주 쓰이는 변수단축
- 36 : 접힌 손가락을 저장할 문자열 초기화

접혀있는 손가락 감지하여 출력하기

```
36 for hand_landmarks in results.multi_hand_landmarks:
37     ...
38     finger_point=hand_landmarks.landmark[8]
39     finger_x=int(finger_point.x*width)
40     finger_y=int(finger_point.y*height)
41     ...
42     mp_drawing.draw_landmarks(
43         frame,
44         hand_landmarks,
45         mp_hands.HAND_CONNECTIONS,
46         line_style,
47         circle_style)
48     #def circle(img, center, radius, color, thickness, lineType)
49     #cv2.circle(frame, (finger_x,finger_y),30,(255,255,255),10)
50     for mcp, tip, comp in compare:
51         tip_dist=(h1[comp].x-h1[tip].x)**2+(h1[comp].y-h1[tip].y)**2
52         mcp_dist=(h1[comp].x-h1[mcp].x)**2+(h1[comp].y-h1[mcp].y)**2
53         folding+="01"[tip_dist>mcp_dist]
```

- 51: 손끝~비교지점거리
- 52: 접힌부분~비교지점거리
- 53: 거리비교 후 접힌경우0, 펼친경우1

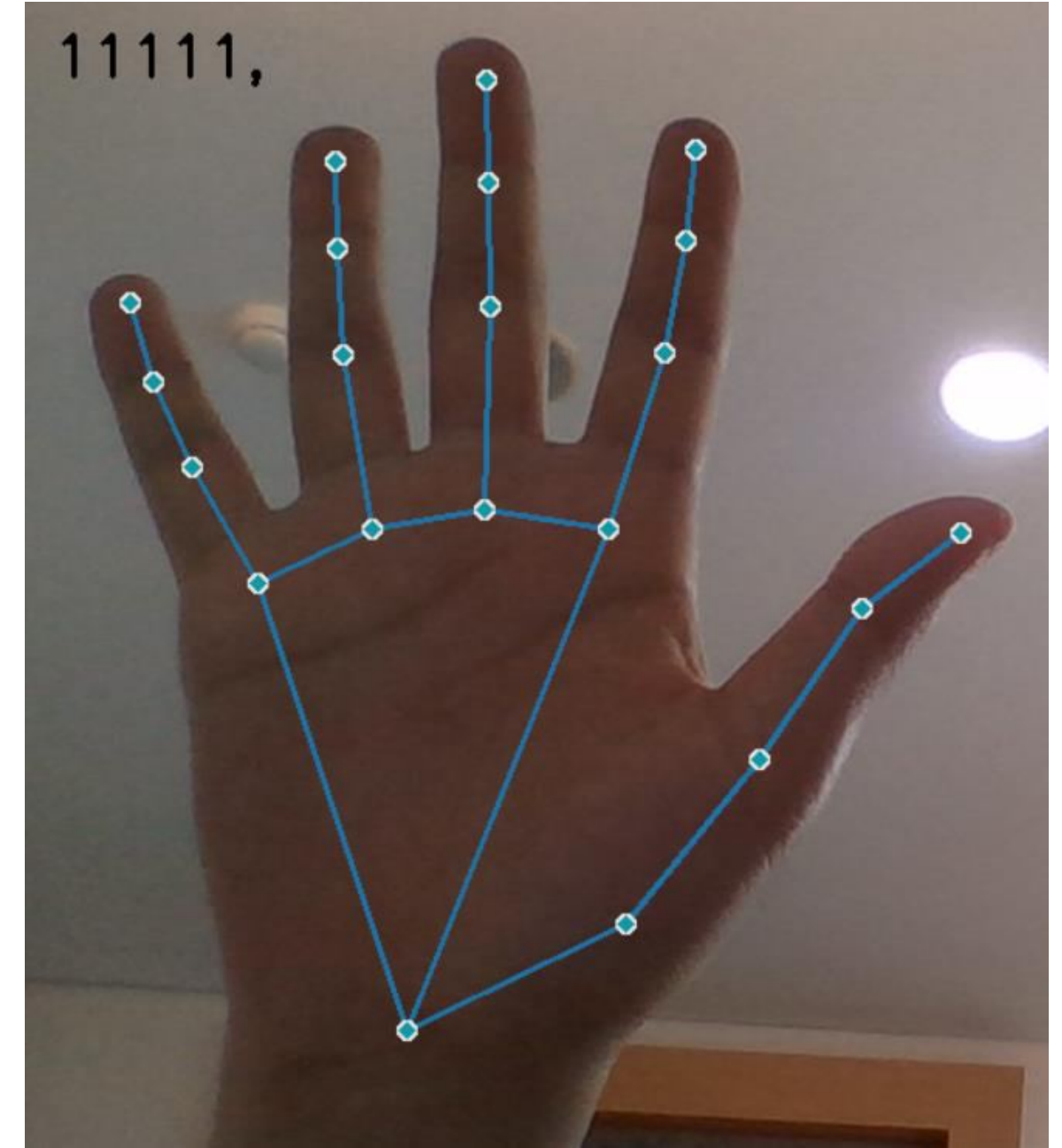
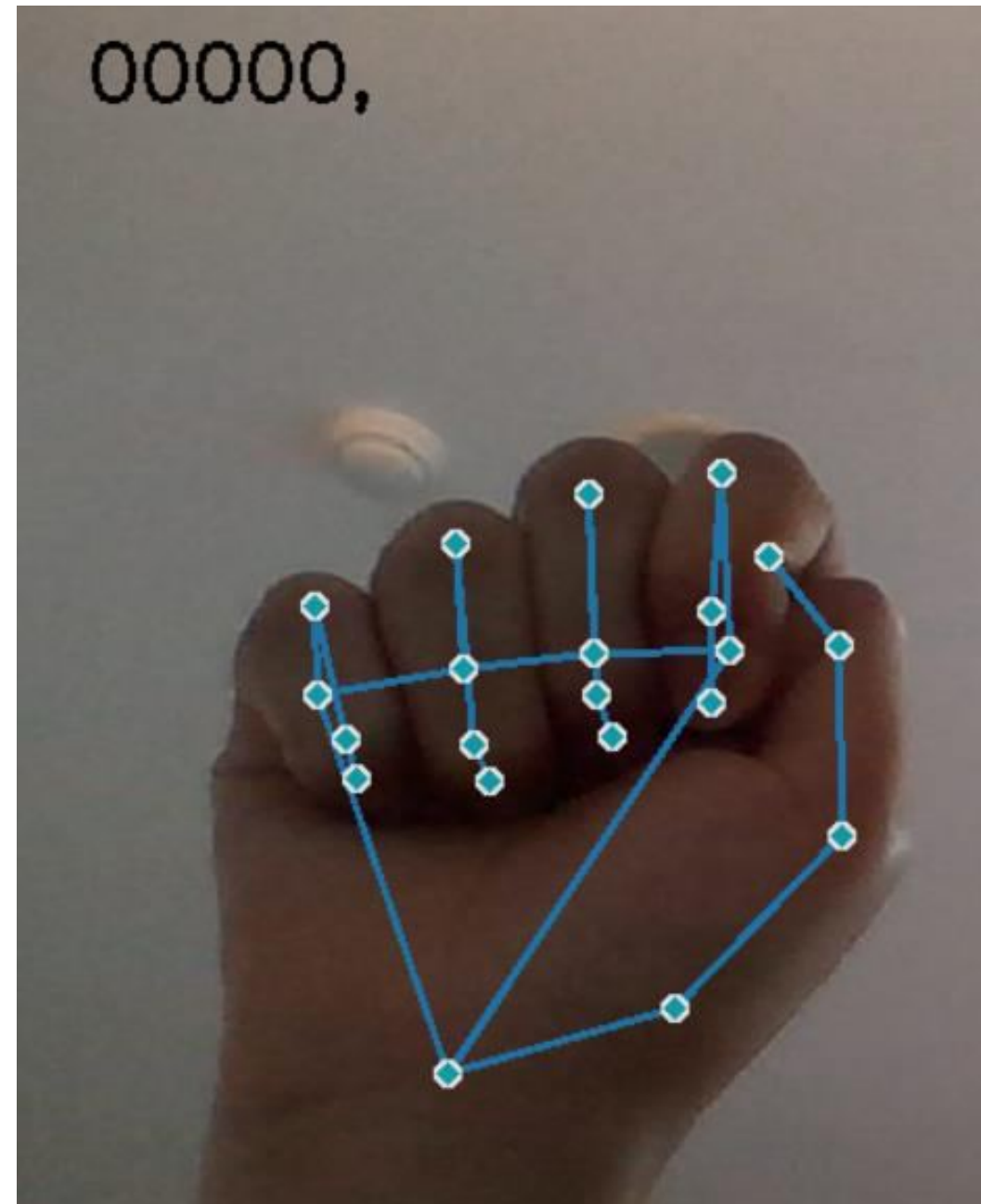
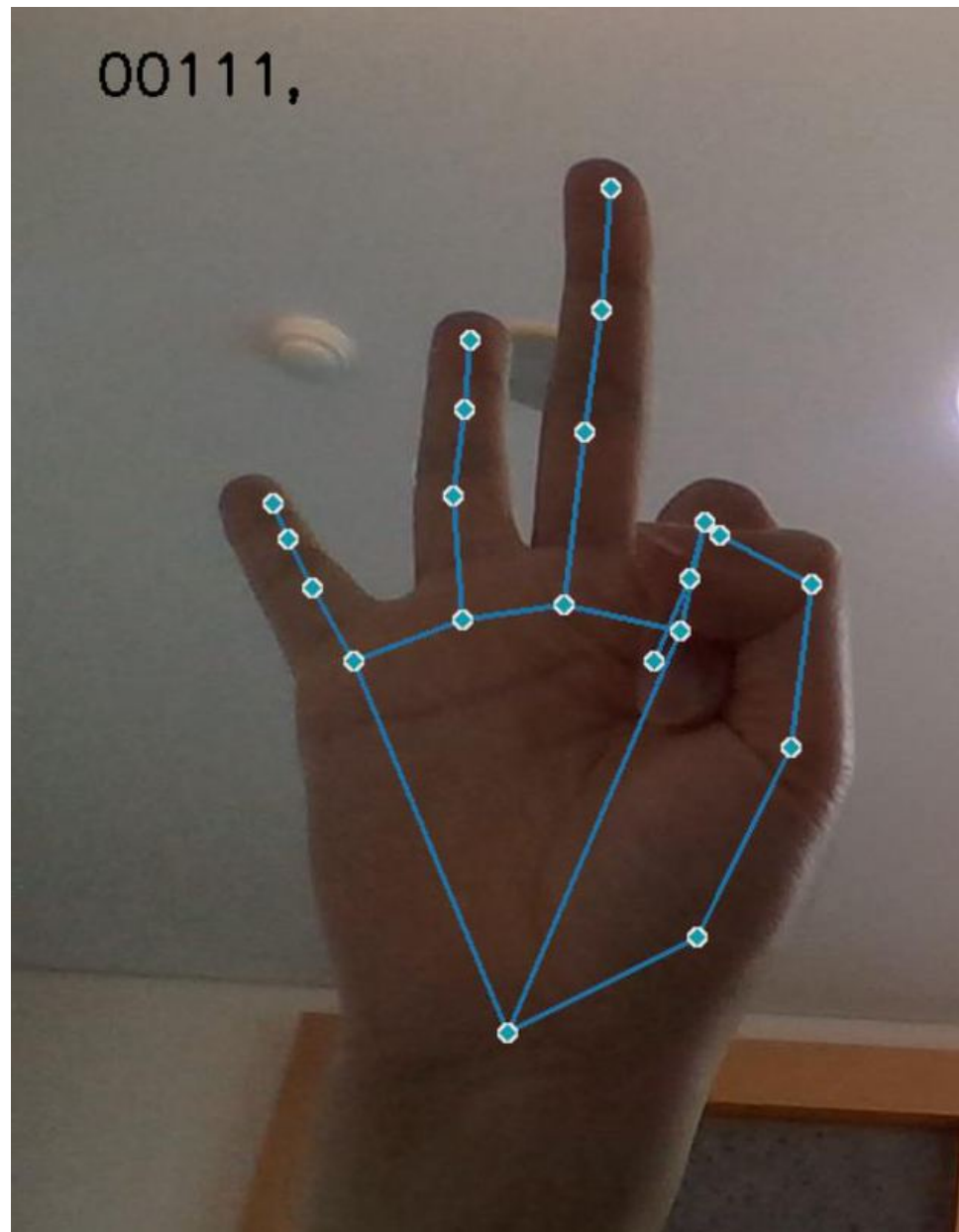
- 비교할 손가락 좌표들을 모두 가져와 점과 점 사이 거리 공식을 이용해 거리 구하기

접혀있는 손가락 감지하여 출력하기

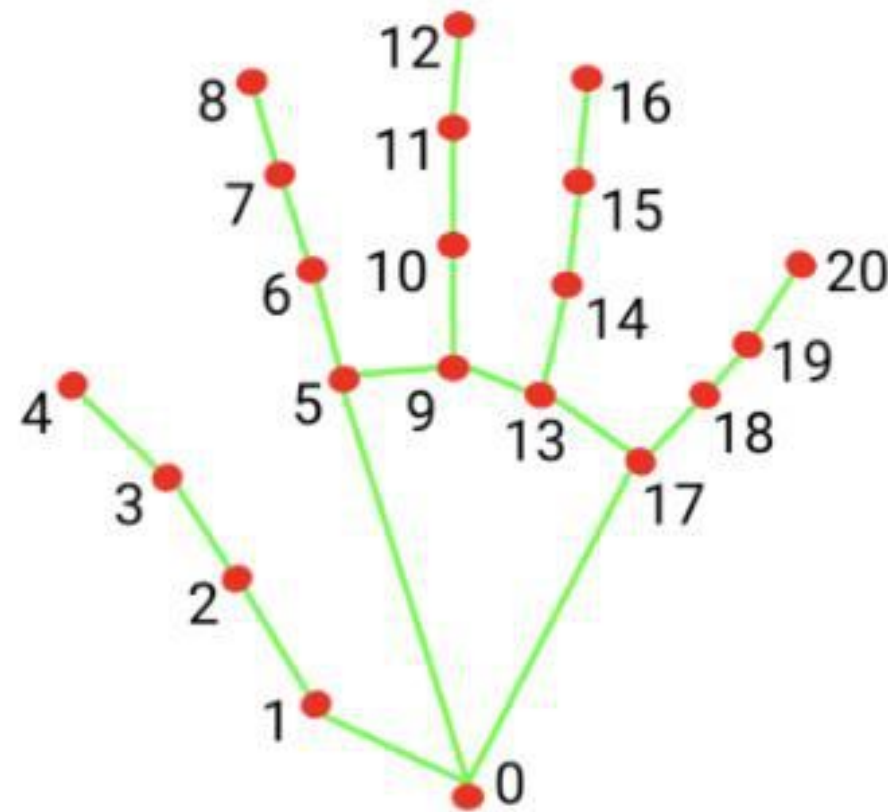
```
50     for mcp, tip, comp in compare:
51         tip_dist=(h1[comp].x-h1[tip].x)**2+(h1[comp].y-h1[tip].y)**2
52         mcp_dist=(h1[comp].x-h1[mcp].x)**2+(h1[comp].y-h1[mcp].y)**2
53         folding+="01"[tip_dist>mcp_dist]
54     folding+=","
55     #cv2.putText(image, text, org(위치), font, fontScale,color, thickness )
56     cv2.putText(
57         frame,
58         folding,
59         (50,50),
60         cv2.FONT_HERSHEY_SIMPLEX,
61         1,
62         (0,0,0),
63         2)
```

- 52: 여러 손일 경우 콤마로 구분하기
- 56-63 : 텍스트로 모든 결과 출력하기

실행 결과



손가락 인식 개선하기



- | | |
|-----------------------|-----------------------|
| 0. WRIST | 11. MIDDLE_FINGER_DIP |
| 1. THUMB_CMC | 12. MIDDLE_FINGER_TIP |
| 2. THUMB_MCP | 13. RING_FINGER_MCP |
| 3. THUMB_IP | 14. RING_FINGER_PIP |
| 4. THUMB_TIP | 15. RING_FINGER_DIP |
| 5. INDEX_FINGER_MCP | 16. RING_FINGER_TIP |
| 6. INDEX_FINGER_PIP | 17. PINKY_MCP |
| 7. INDEX_FINGER_DIP | 18. PINKY_PIP |
| 8. INDEX_FINGER_TIP | 19. PINKY_DIP |
| 9. MIDDLE_FINGER_MCP | 20. PINKY_TIP |
| 10. MIDDLE_FINGER_PIP | |

```
23 compare = [(2,4,17),(5,8,0),(9,12,0),(13,16,0),(17,20,0)]
24 #카메라가 동작하는 동안 계속하여 반복
25 while True:
```

- 손목이 가려지는 경우 인식률이 떨어진다는 단점
- 더 좋은 방법은?

수동 좌표 계산이 아니 인공지능 학습으로

- Mediapipe내에 Hand Gesture라는 모듈로 제스처 인식 가능
- 제스처 인식의 경우 손모양을 Tensorflow로 학습시킴
- 머신 러닝으로 학습된 결과를 사용하므로 조금 더 정확

양손 구분하기

```
1  ✓ import cv2
2  import mediapipe as mp
3  mp_hands=mp.solutions.hands
4  mp_drawing=mp.solutions.drawing_utils
5  hands=mp_hands.Hands(max_num_hands=3)
```

- 5 : 손인식 개수 3개로 설정

양손 구분하기

```
25 while True:
26     # 비디오 스트림(카메라) 캡처
27     ret, frame=cap.read()
28     frame=cv2.flip(frame,1)
29     if not ret : break #읽어오는데 실패했다면 종료하기
30     image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
31     results=hands.process(image)
32     if results.multi_handedness:
33         for handeness in results.multi_handedness:
34             cv2.putText(
35                 frame,
36                 handeness.classification[0].label,
37                 (50,50),
38                 cv2.FONT_HERSHEY_PLAIN,
39                 2,
40                 (255,255,255),
41                 1)
```

- 32-41 : 손 인식 결과 값 안에는 어떤 쪽의 손인지 판별하는 정보도 담겨있음
- 34-41 : 어떤 손인지 글씨로 출력하는 코드

실행 결과



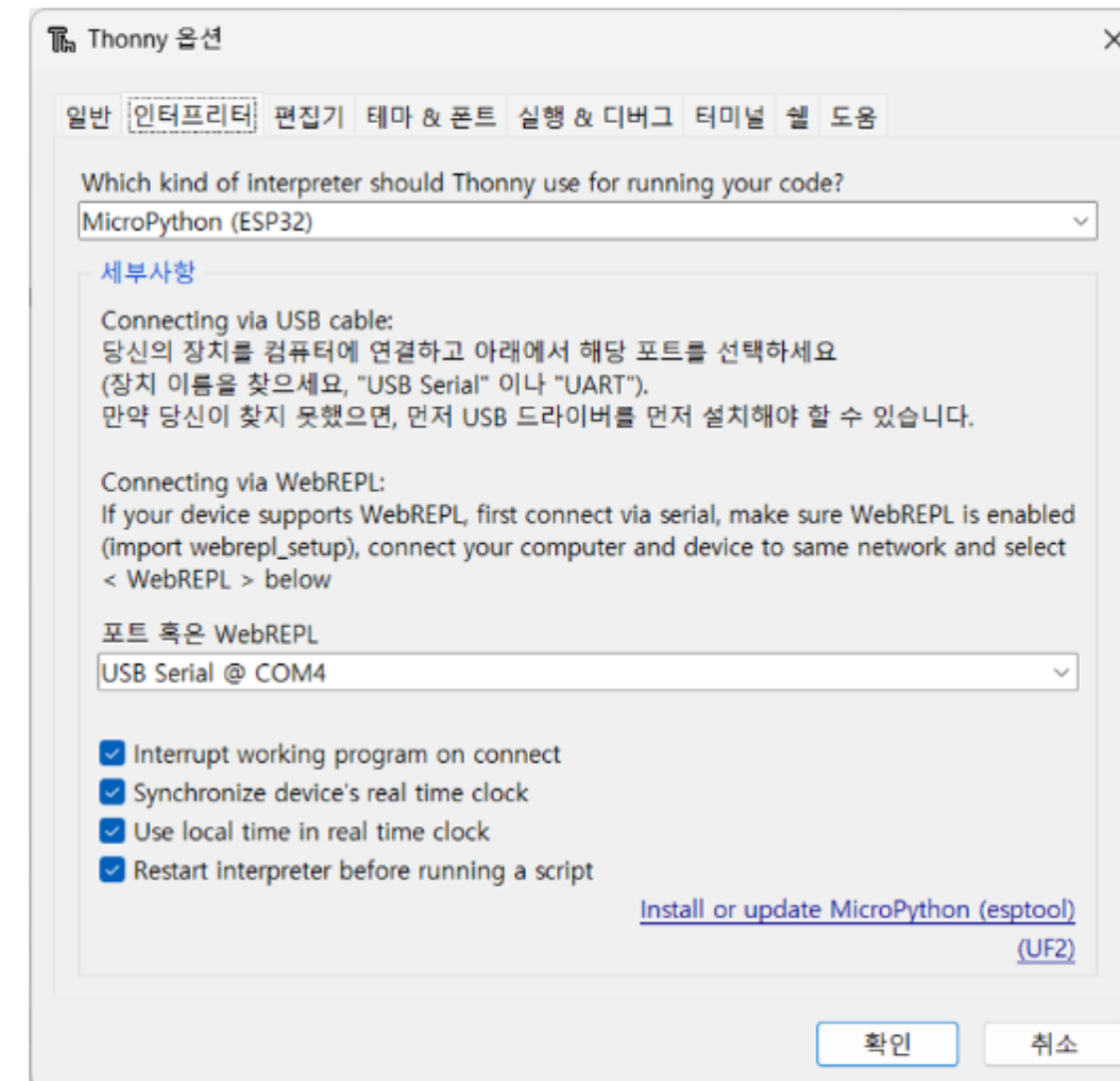
토니파이션

개발 환경 만들기 - Thonny 설치하기

Thonny 설치하기

- <https://thonny.org/> 접속 후 다운로드
- 마이크로파이썬을 따로 설치하지 않고 편하게 사용할 수 있음
- 마이크로프로세서 보드에 쉽게 연결할 수 있음

1. 설치 후 실행 한국어, Standard로 설정 후 Let's Go.
2. ESP32 USB 연결
3. 도구(Tools) ➡ 옵션 ➡ 인터프리터 탭 ➡ MicroPython (ESP32)로 설정
4. 포트 USB Serial @COM(숫자) 선택



ESP32 기본 펌웨어 설치하기

보드 인식 안되는 경우: ESP32 호환 드라이버 설치하기

- USB에 꽂아도 인식이 안되는 경우에만 진행
- 보드를 꽂은 상태에서 아래의 파일(압축 해제 후)을 설치

 CH34x_Install_Windows_v3_4.zip 189.1KB

ESP32 기본 펌웨어 설치하기

기존 파일 업로드하기(모듈)

1. 보기 > 파일 ✓체크하기
2. 이 컴퓨터 클릭 후 가져올 파일 경로 찾기(.py 확장자만 가능)
3. 파일 오른쪽 클릭 ➡ / 에 업로드 클릭
 - 에러가 뜨면서 업로드가 안될 경우 ●버튼을 클릭 후 진행해볼 것

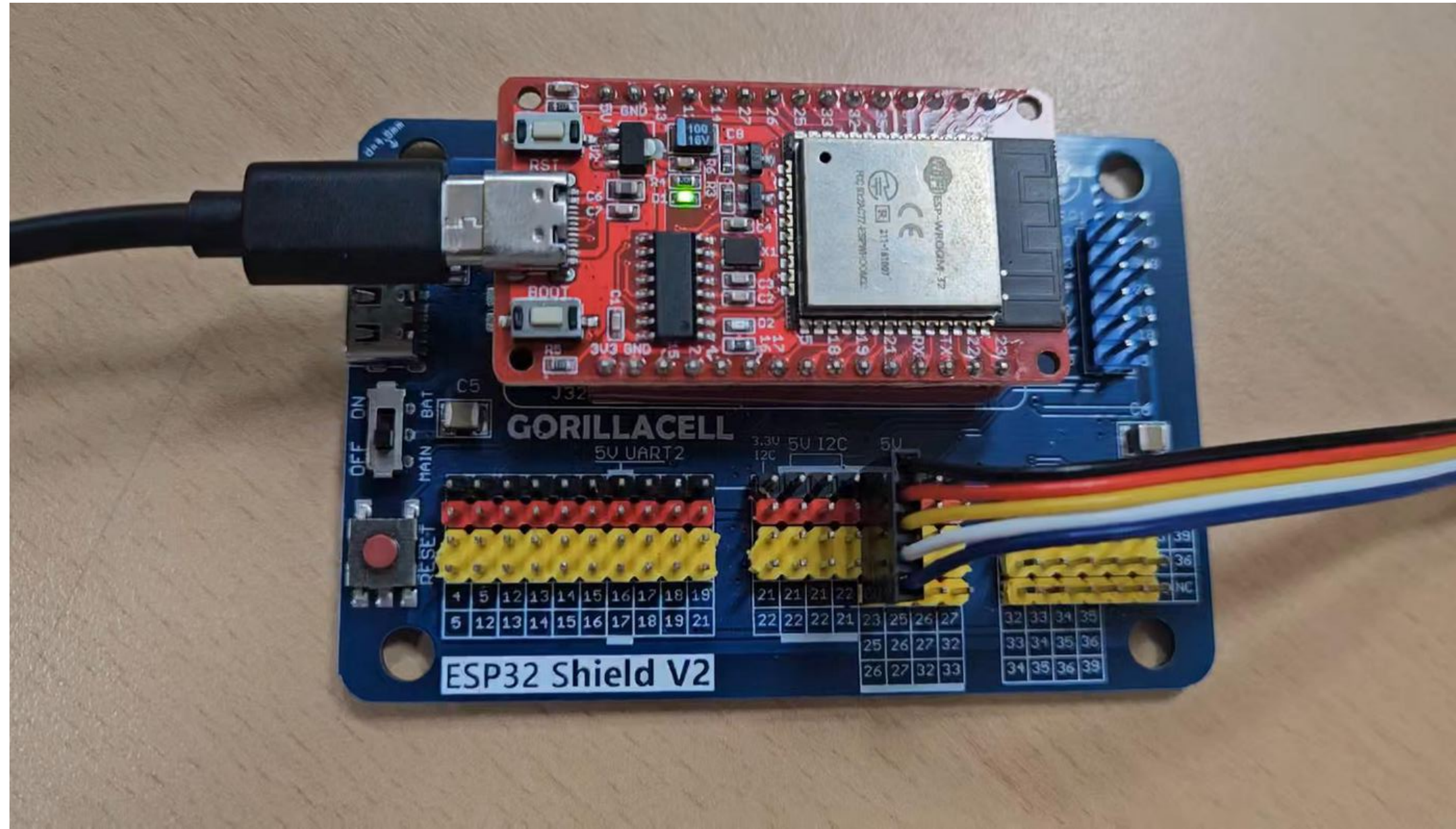
ESP32 연결



Light turn on off

```
1 from machine import Pin
2 from time import sleep #delay를 위한 sleep
3
4 led=Pin(2,Pin.OUT) #2번 pin인데 out pin
5 # 프롬프트 창에 led.on()을 하게되면 esp32 아래에서 두번째줄에 불이 들어오게된다.
6 # led.off()하게 되면 불이꺼지게 된다.(이거는 shell창)
7
8 # 만약 shell 창이 아닌 바로 명령을 하고 싶은 경우
9 while True:
10     led.on()
11     sleep(0.5)
12     led.off()
13     sleep(0.5) # 이렇게 하게되면 led가 0.5초 간격으로 꺼졌다 켜졌다 한다. |
```

Light turn on off



Led_rgb(센서)



Led_rgb(센서)

```
1 from machine import Pin
2 from time import sleep
3
4 r=Pin(12, Pin.OUT)
5 g=Pin(5, Pin.OUT)
6 b=Pin(4, Pin.OUT)
7
8 led=[r,g,b]
9 while True:
10     for x in range(3):
11         print(x)
12         led[x].on()
13         sleep(1)
14         led[x].off()
15         sleep(1)
16
```


LED_RGB 작동영상



RGB LED 색상조합 및 밝기 제어하기

```
1 #사용할 라이브러리 추가
2 from machine import Pin
3 from machine import PWM
4 from time import sleep
5
6 #핀 설정
7 r = Pin(12, Pin.OUT)
8 g = Pin(5, Pin.OUT)
9 b = Pin(4, Pin.OUT)
10
11 #핀 설정
12 r = PWM(Pin(12))
13 g = PWM(Pin(5))
14 b = PWM(Pin(4))
15
16 #펄스를 0으로 설정
17 r.duty(0)
18 g.duty(0)
19 b.duty(0)
```

이 PC에 저장됨

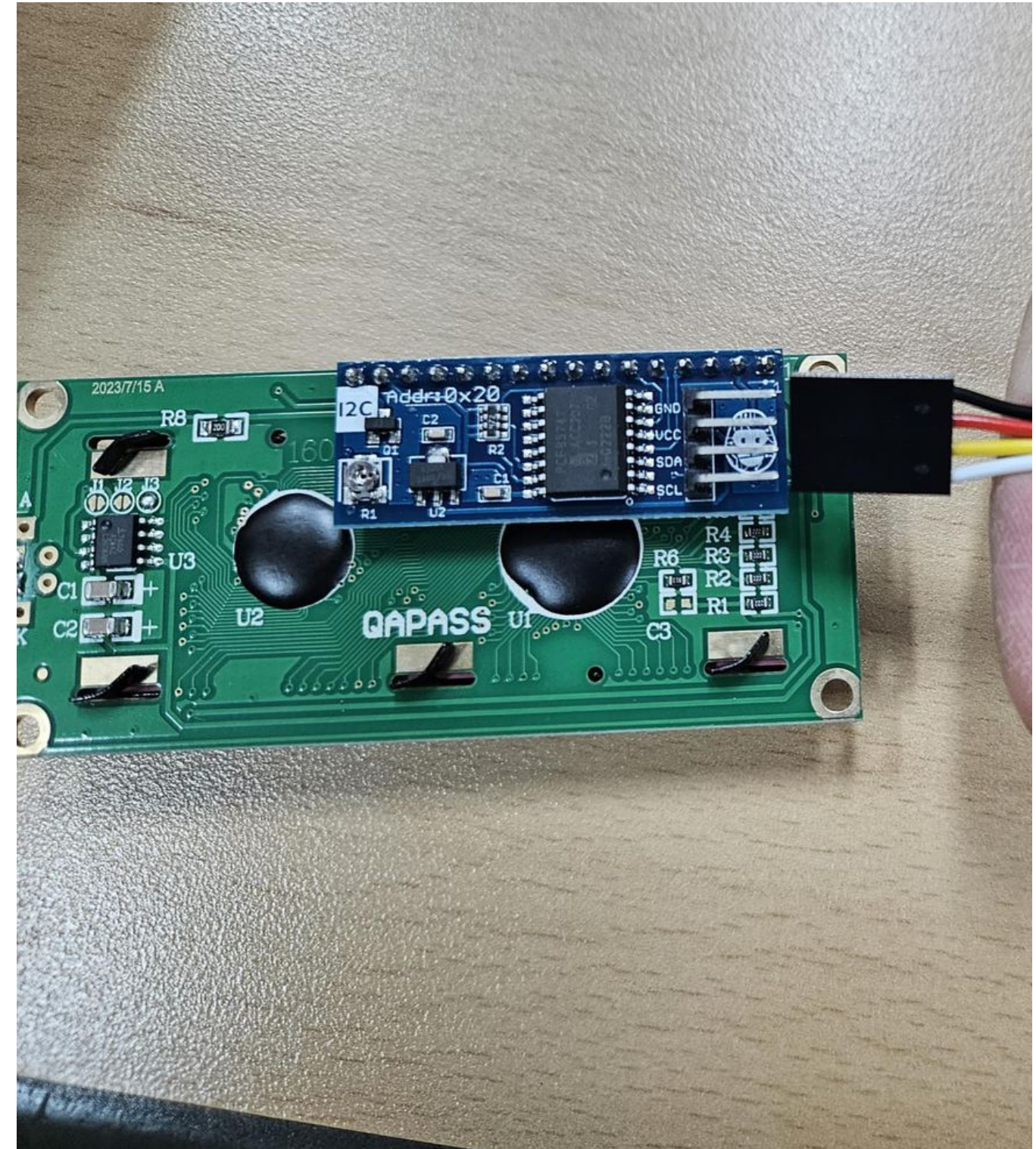
RGB LED 색상조합 및 밝기 제어하기

```
21 #듀티비 변화 시키기
22 while True:
23     for i in range(1024):
24         r.duty(i)
25         g.duty(0)
26         b.duty(0)
27         sleep(0.001)
28     for i in range(1023, -1, -1): #Range is (start, end-1, increment/decrement)
29         r.duty(i)
30         g.duty(0)
31         b.duty(0)
32         sleep(0.001)
```

RGB LED 색상조합 및 밝기 제어(영상)



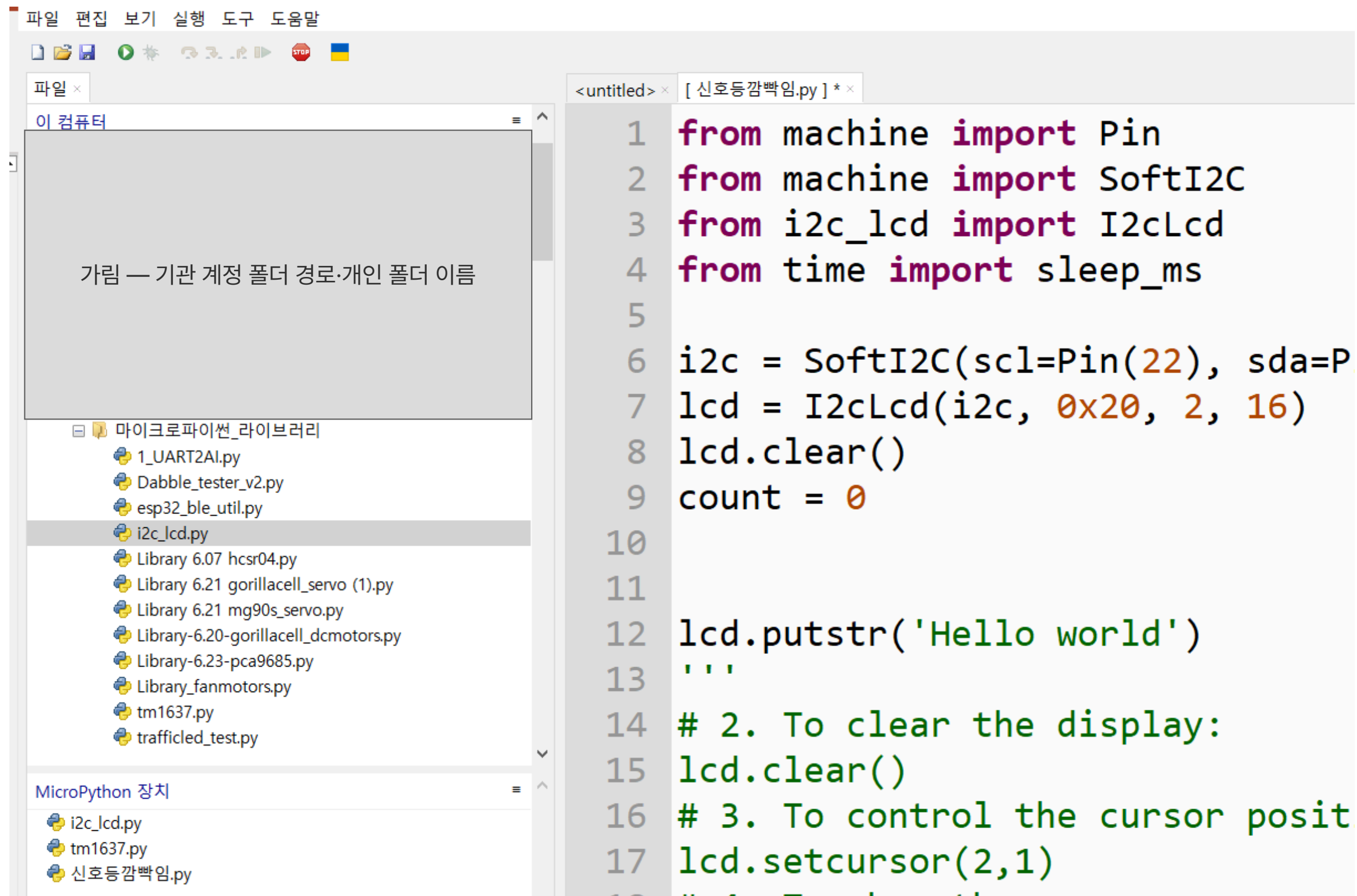
LCD 피지컬 교구



LCD 피지컬 교구



LCD- 라이브러리 업로드



LCD- 코드

```
1 from machine import Pin
2 from machine import SoftI2C
3 from i2c_lcd import I2cLcd
4 from time import sleep_ms
5
6 i2c = SoftI2C(scl=Pin(22), sda=Pin(21), freq=400000)
7 # freq는 I2c 통신의 주파수 설정.
8 lcd = I2cLcd(i2c, 0x20, 2, 16)
9 #0x20은 LCD 디스플레이 I2C주소
10 # 2행, 16열. 총 32개의 문자 표시.
11 lcd.clear()
12 count = 0
```

LCD 코드

```
14 lcd.putstr('Hello world')
15 sleep(1)
16 # 2. To clear the display:
17 lcd.clear()
18 sleep(1)
19
20 # 3. To control the cursor position:
21 lcd.setCursor(2,1)
22 sleep(1)
23
24 # 4. To show the cursor:
25 lcd.show_cursor()
26 sleep(1)
```

LCD 코드

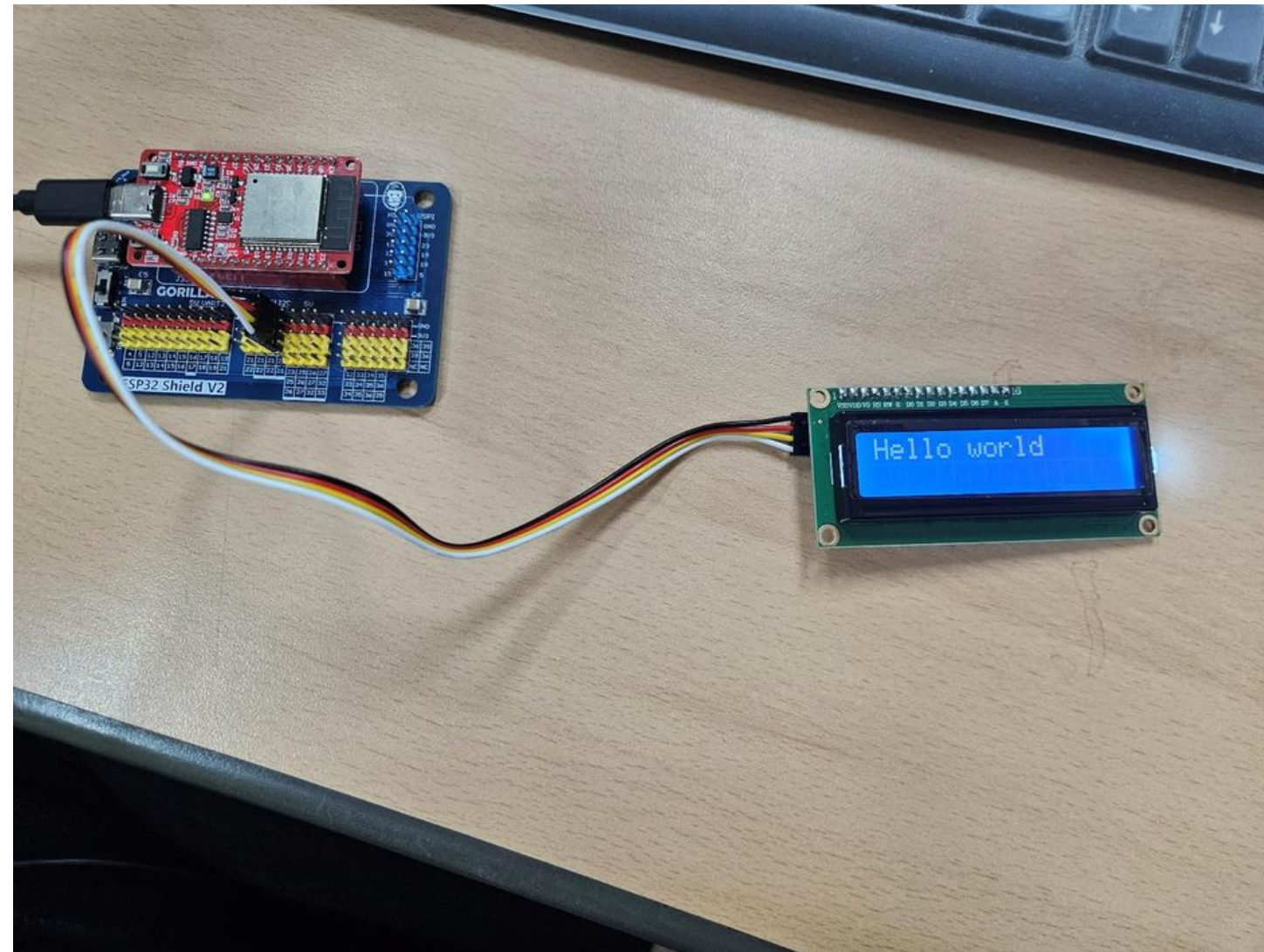
```
28 # 5. To hide the cursor:
29 lcd.hide_cursor()
30 sleep(1)
31
32 # 6. To set the cursor to blink:
33 lcd.blink_cursor_on()
34 sleep(1)
35
36 # 7. To stop the cursor on blink:
37 lcd.blink_cursor_off()
38 sleep(1)
```

```
40 # 8. To hide the currently displayed character:
41 lcd.display_off()
42 sleep(1)
43
44 # 9. To show the currently hidden character:
45 lcd.display_on()
46 sleep(1)
47
48 # 10. To turn off the backlight:
49 lcd.backlight_off()
50 sleep(1)
```

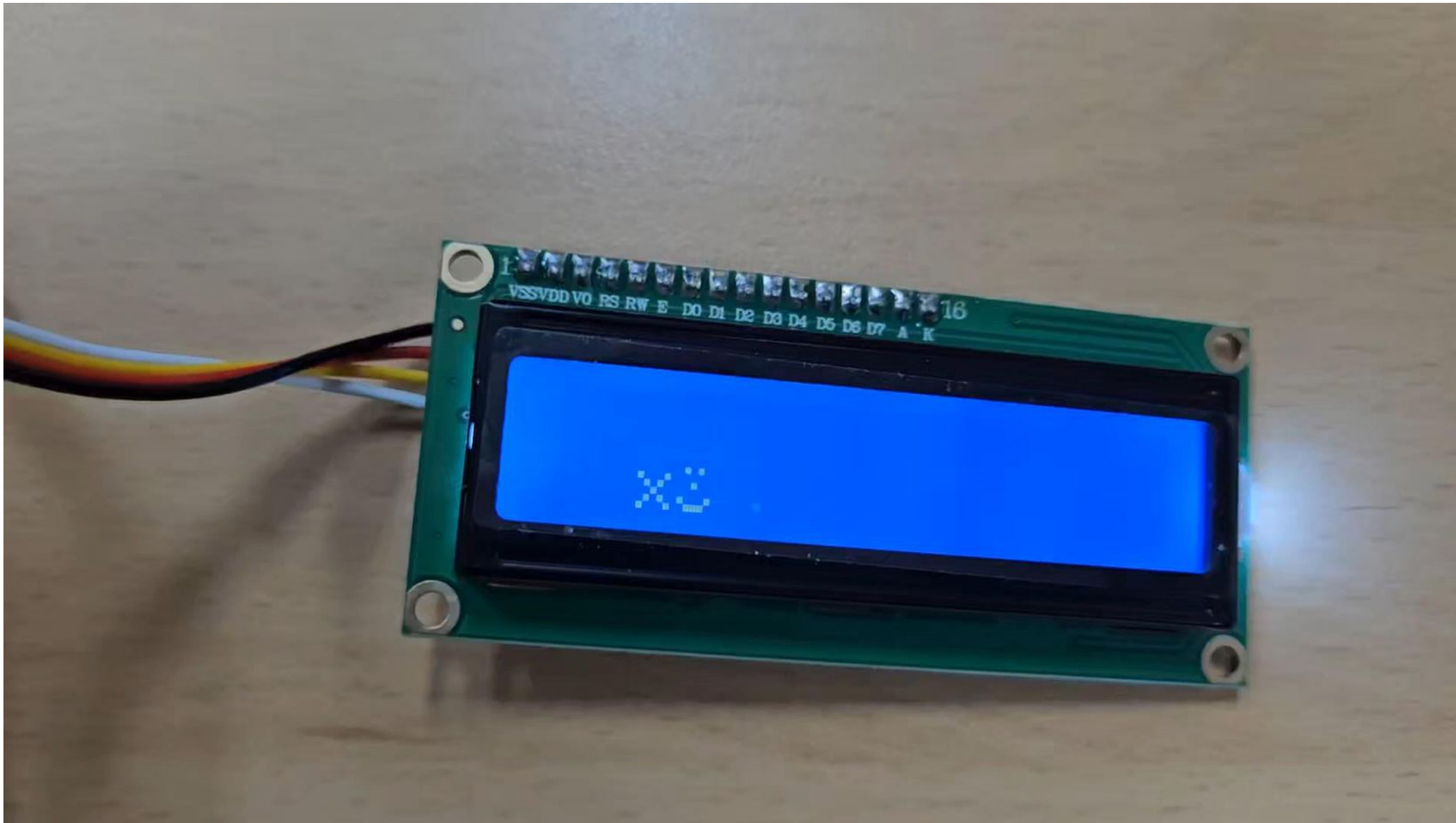

LCD 코드

```
52 # 11. To turn ON the backlight:
53 lcd.backlight_on()
54 sleep(1)
55
56 # 12. To print a single character:
57 lcd.putchar('x')
58 sleep(1)
59
60 # 13. To print a custom character:
61 happy = bytearray([0x00,0x0A,0x00,0x04,0x00,0x11,0x0E,0x00])
62 lcd.custom_char(0, happy)
63 lcd.putchar(chr(0))
```

LCD 영상



LCD 영상



LCD 카운트 값 증가

The screenshot displays the MicroPython IDE interface. On the left, the file explorer shows a project named '이 컴퓨터' (This Computer) with a subfolder '마이크로파이썬 라이브러리' (MicroPython Library) containing various files. The 'boot.py' file is selected. The main editor window shows a Python script for an LCD counter. The script imports necessary modules and initializes an I2C LCD. It then enters a loop that prints the counter value to the LCD and increments it by 1 every 1000 milliseconds. The output window at the bottom shows the current counter value as 3.

```
1 from machine import Pin
2 from machine import SoftI2C
3 from i2c_lcd import I2cLcd
4 from time import sleep_ms
5
6 i2c = SoftI2C(scl=Pin(22), sda=Pin(21), freq=400000)
7 lcd = I2cLcd(i2c, 0x20, 2, 16)
8 lcd.clear()
9 count = 0
10 while True:
11     print('Counter: ', count)
12     lcd.clear()
13     lcd.putstr(f'Counter: {count}')
14     count = count + 1
15     sleep_ms(1000)
16
17
18
19
```

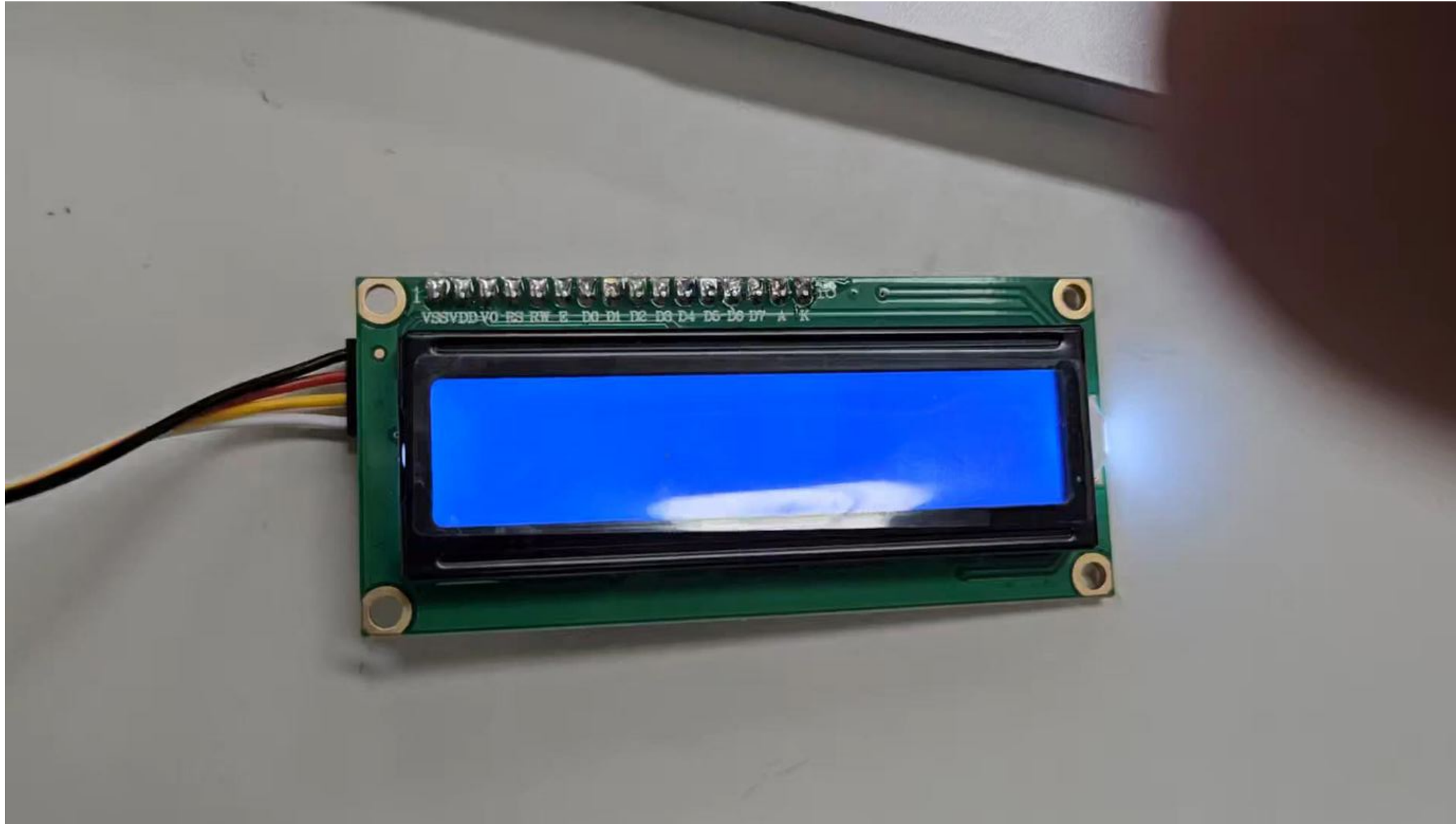
Counter: 3

LCD 카운트 값 증가

<untitled> × [boot.py] ×

```
1 from machine import Pin
2 from machine import SoftI2C
3 from i2c_lcd import I2cLcd
4 from time import sleep_ms
5
6 i2c = SoftI2C(scl=Pin(22), sda=Pin(21), freq=400000)
7 lcd = I2cLcd(i2c, 0x20, 2, 16)
8 lcd.clear()
9 count = 0
10 while True:
11     print('Counter: ',count)
12     lcd.clear()
13     lcd.putstr(f'Counter: {count}')
14     count = count + 1
15     sleep_ms(1000)
```

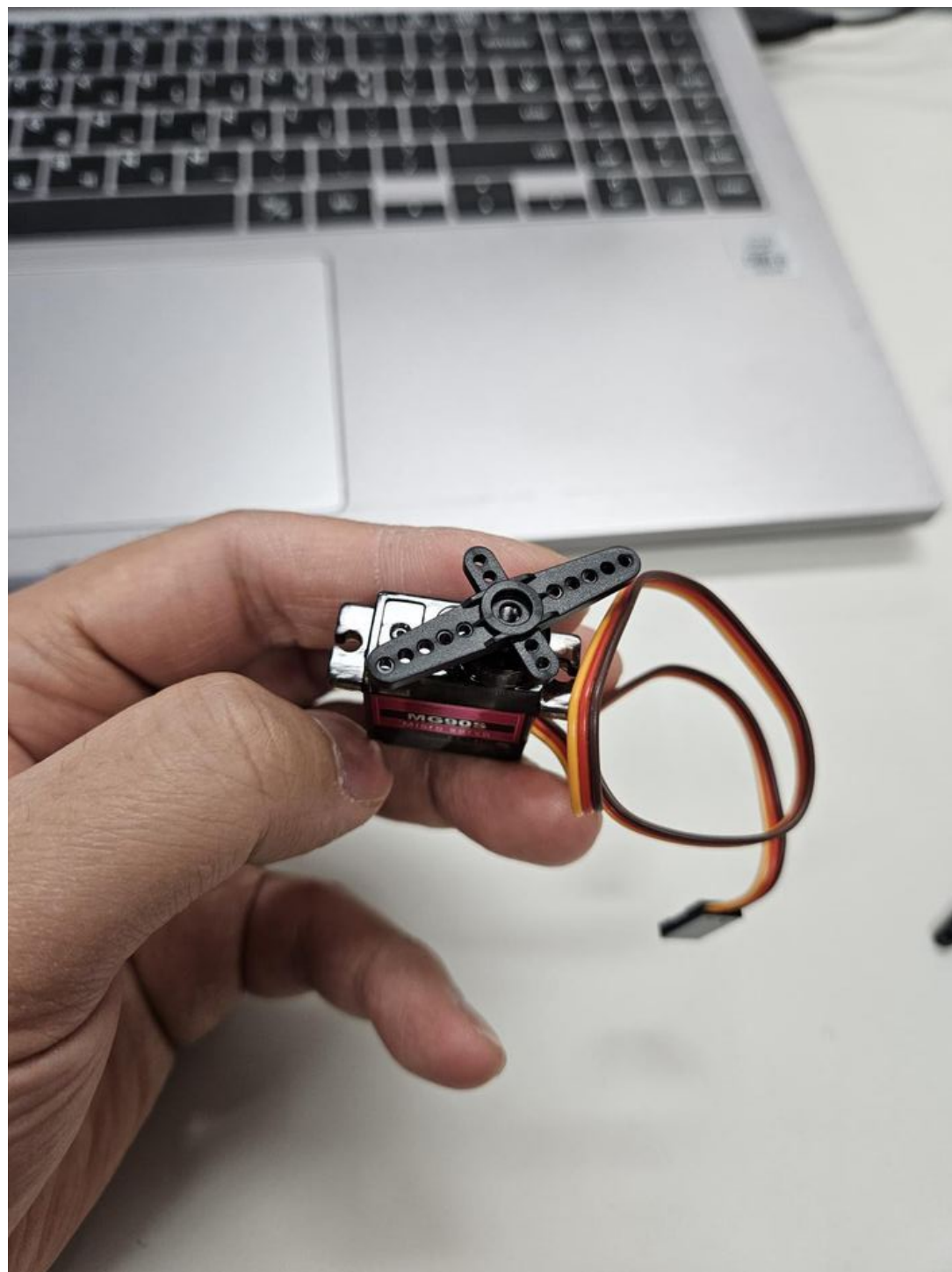

LCD 카운트 값 증가(실행)



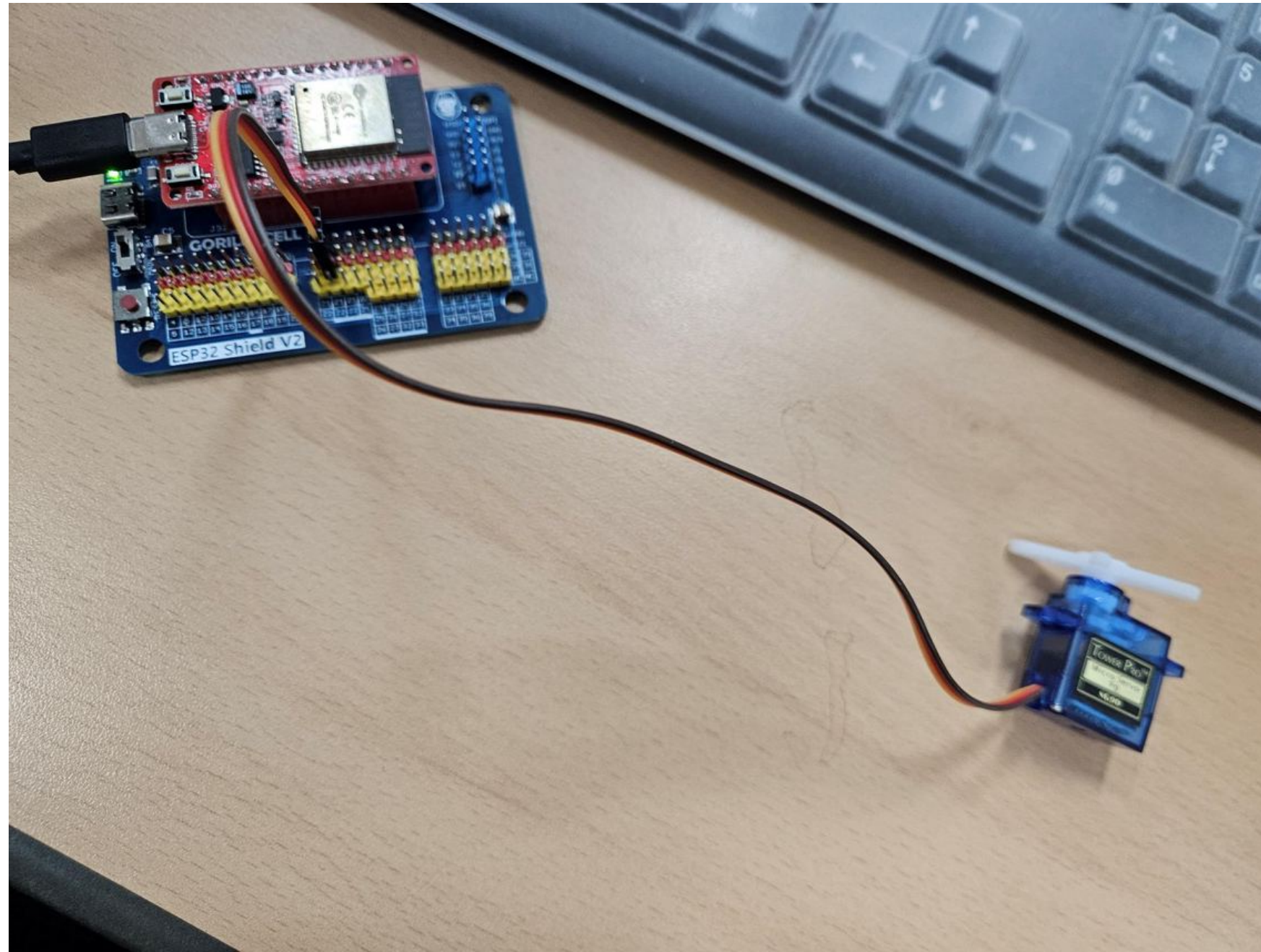
서보 모터(피지컬 교구)



서보 모터(피지컬 교구)



서보 모터(피지컬 교구)



서보 모터 라이브러리 업로드

-  Library-6.23-pca9685.py
-  Library_fanmotors.py
-  mg90s_servo.py
-  tm1637.py
-  trafficked_test.py
-  UART2AI_TM.py
-  working_test.py
-  led_rgb.py

MicroPython 장치

-  boot.py
-  mg90s_servo.py

서보 모터 코드

```
1 from machine import Pin
2 from machine import PWM
3 from time import sleep_ms
4 from mg90s_servo import MG90S_SERVO
5
6 se=MG90S_SERVO(signal_pin=32)
7 #se.rotate(90)
8 #se.rotate(180)
9 #se.rotate(0) 해보기
10
11 while True:
12     se.rotate(0)
13     sleep_ms(500)
14     se.rotate(90)
15     sleep_ms(500)
```

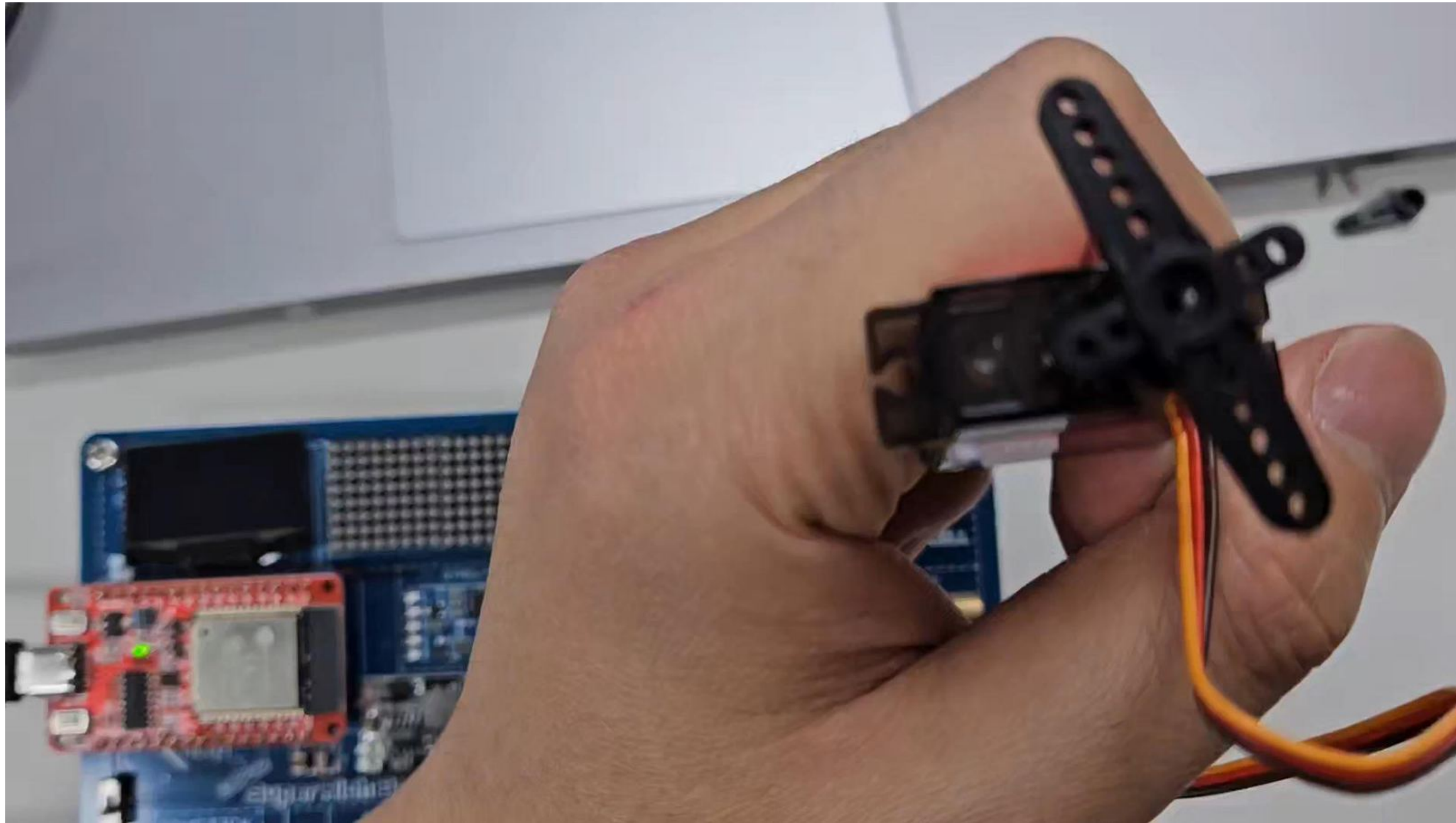
서보 모터 실행 영상



서보 모터 증가

```
<untitled> x [ boot.py ] ^ x
1 from machine import Pin
2 from machine import PWM
3 from time import sleep_ms
4 from mg90s_servo import MG90S_SERVO
5
6 se=MG90S_SERVO(signal_pin=32)
7
8 for angle in range(0,181,10):
9     se.rotate(angle)
10    print(angle)
11    sleep_ms(500)
12
13 for angle in range(180,-1,-10):
14     se.rotate(angle)
15     print(angle)
16     sleep_ms(500)
```

서보 모터 증가(실행 영상)



블루투스 통신

블루투스 통신(기본) 값 주고 받기

값을 던짐

가림 — 학급 게시물 화면

가림 — 학급 게시물 화면

값을 던진다 (영상처리 이용해서 값을 던진다.)

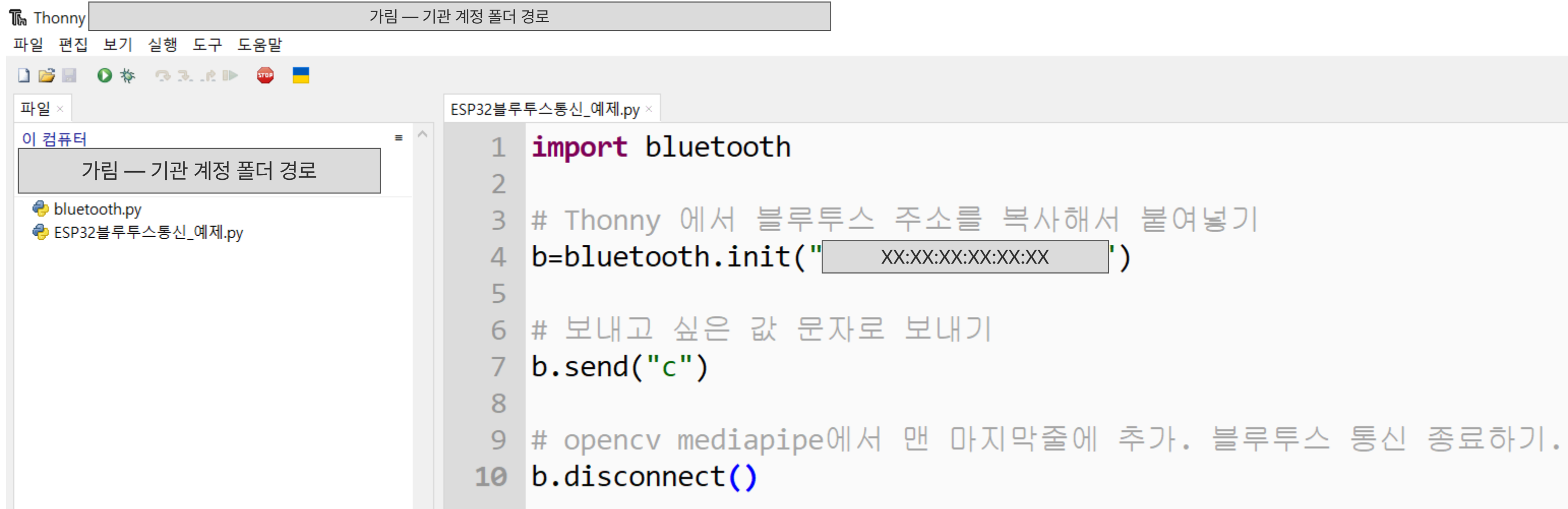
-우측 하단 local python으로 설정

두개의 파일을 다운로드 후 하나의 폴더에 넣기

값을 받는다(피지컬 교구를 이용해서 값을 받는다.)

-우측 하단 ESP32 로 설정

(값을 던짐)Thonny (폴더 불러오기)



값을 던진다 (영상처리 이용해서 값을 던진다.)

-우측 하단 local python으로 설정

값을 받는다(피지컬 교구를 이용해서 값을 받는다.)

-우측 하단 ESP32 로 설정

(값을 받음) Thonny

가림 — 학급 게시물 화면

가림 — 학급 게시물 화면

가림 — 학급 게시물 화면

값을 던진다 (영상처리 이용해서 값을 던진다.)

-우측 하단 local python으로 설정

값을 받는다(피지컬 교구를 이용해서 값을 받는다.)

-우측 하단 ESP32 로 설정

ESP32BLE.py를 마이크로파이썬 장치에 업로드 후
Thonny 코드 작성하기

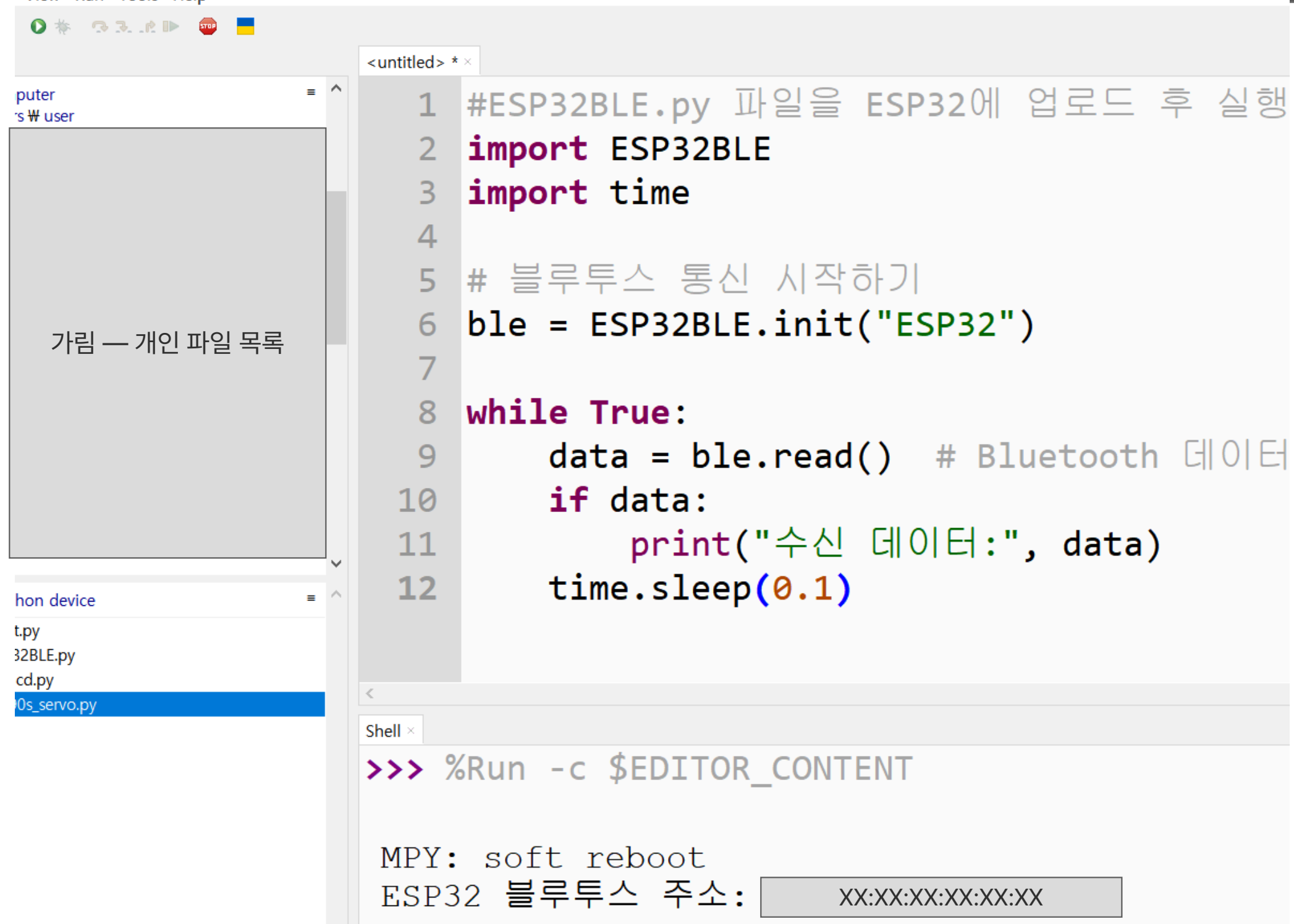
(값을 받음) Thonny(boot.py)에 저장

```
1 #ESP32BLE.py 파일을 ESP32에 업로드 후 실행
2 import ESP32BLE
3 import time
4
5 # 블루투스 통신 시작하기
6 ble = ESP32BLE.init("ESP32")
7
8 while True:
9     data = ble.read() # Bluetooth 데이터 읽기
10    if data:
11        print("수신 데이터:", data)
12    time.sleep(0.1)
```

값을 던진다 (영상처리 이용해서 값을 던진다.)
-우측 하단 local python으로 설정

값을 받는다(피지컬 교구를 이용해서 값을 받는다.)
-우측 하단 ESP32 로 설정

Thonny 재생 후 물리적 주소 얻기



The screenshot shows the Thonny IDE interface. On the left, there is a file explorer with a tree view showing a project named 'hon device' containing files 't.py', '32BLE.py', 'cd.py', and '0s_servo.py'. The main editor window displays a Python script for ESP32 BLE communication. The script includes comments in Korean and code for importing ESP32BLE and time modules, initializing a BLE object, and entering a loop to read and print data from a Bluetooth device. The bottom panel shows a terminal window with the command prompt 'Shell' and the command '%Run -c \$EDITOR_CONTENT'. The terminal output shows 'MPY: soft reboot' and 'ESP32 블루투스 주소: XX:XX:XX:XX:XX:XX'.

```
<untitled> * x
1 #ESP32BLE.py 파일을 ESP32에 업로드 후 실행
2 import ESP32BLE
3 import time
4
5 # 블루투스 통신 시작하기
6 ble = ESP32BLE.init("ESP32")
7
8 while True:
9     data = ble.read() # Bluetooth 데이터
10    if data:
11        print("수신 데이터:", data)
12    time.sleep(0.1)
```

```
Shell x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
ESP32 블루투스 주소: XX:XX:XX:XX:XX:XX
```

값을 던진다 (영상처리 이용해서 값을 던진다.)

-우측 하단 local python으로 설정

값을 받는다(피지컬 교구를 이용해서 값을 받는다.)

-우측 하단 ESP32 로 설정

Thonny 물리적 주소를 (값을 던짐) 코드에 붙여 넣기

ESP32블루투스통신_예제.py ×

```
1 import bluetooth
2
3 # Thonny 에서 블루투스 주소를 복사해서 붙여넣기
4 b=bluetooth.init("XX:XX:XX:XX:XX:XX")
5
6 # 보내고 싶은 값 문자로 보내기
7 b.send("c")
8
9 # opencv mediapipe에서 맨 마지막줄에 추가. 블루투스 통신 종료하기.
10 b.disconnect()
```

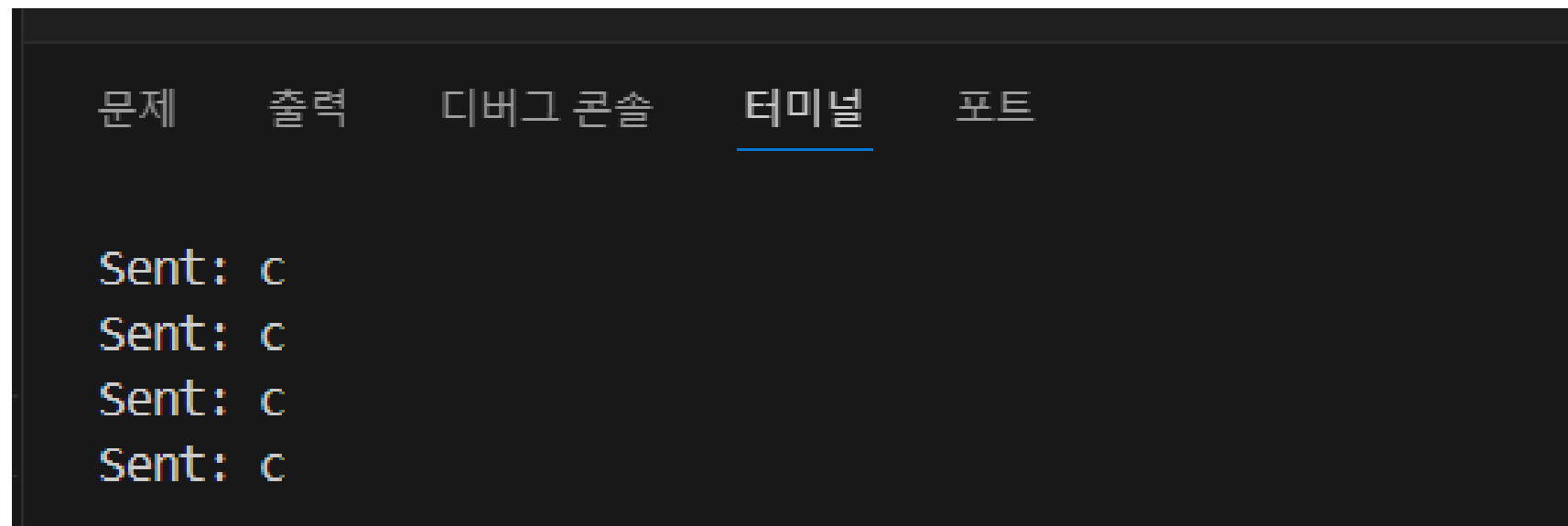
값을 던진다 (영상처리 이용해서 값을 던진다.)

-우측 하단 local python으로 설정

값을 받는다(피지컬 교구를 이용해서 값을 받는다.)

-우측 하단 ESP32 로 설정

Vscode 재생하면 Thonny로 값 보내지게 된다.



A screenshot of the VS Code terminal interface. The terminal has tabs for '문제', '출력', '디버그 콘솔', '터미널' (selected), and '포트'. The output shows four lines of 'Sent: c'.

```
문제  출력  디버그 콘솔  터미널  포트

Sent: c
Sent: c
Sent: c
Sent: c
```

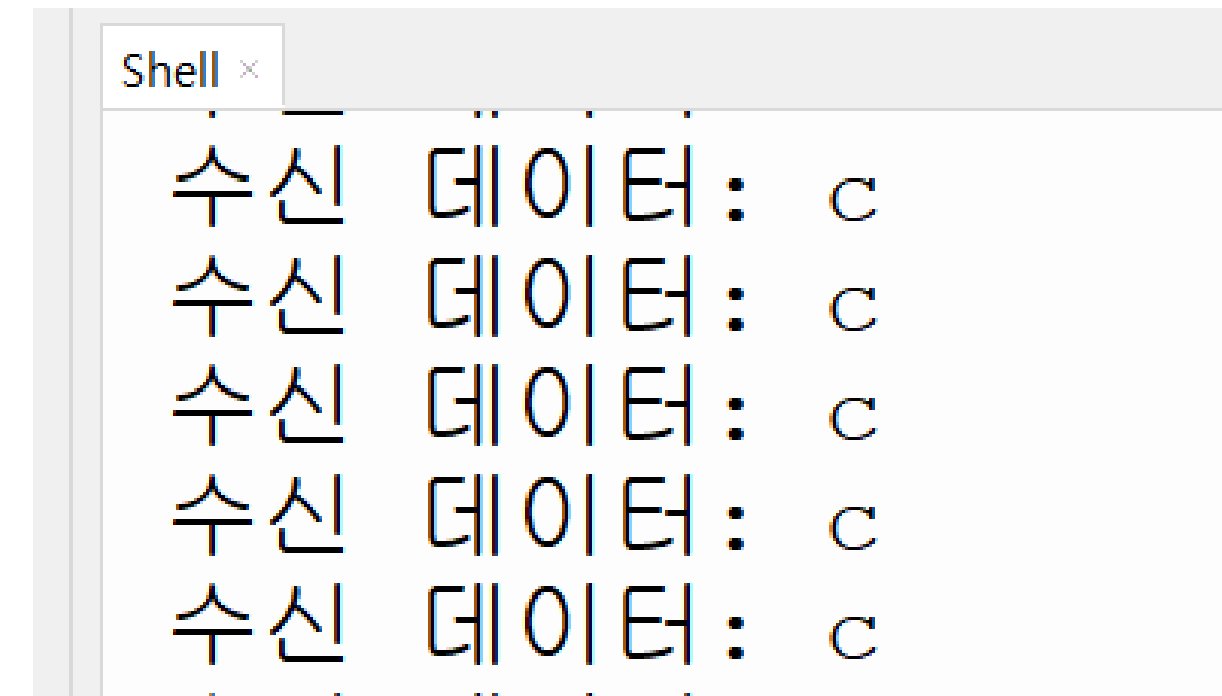
값을 던지는 부분 : VSCODE

값을 받는부분 : Thonny

에디터 두개 사용해서 한눈에 보기 쉽게 표기

우리는 에디터 두개가 아닌 thonny 하나만 사용하기 때문에
값을 받는 부분의 코드를 boot.py에 업로드해서 전원만 들어오면 프로그램이 계속 돌아가게 만들고

값을 던지는 부분을 Thonny를 이용해서 던진다.



A screenshot of a shell window titled 'Shell x'. It displays five lines of '수신 데이터: c'.

```
Shell x

수신 데이터: c
수신 데이터: c
수신 데이터: c
수신 데이터: c
수신 데이터: c
```

블루투스 통신(응용) 값 주고 받기(손가락으로 RGB제어)

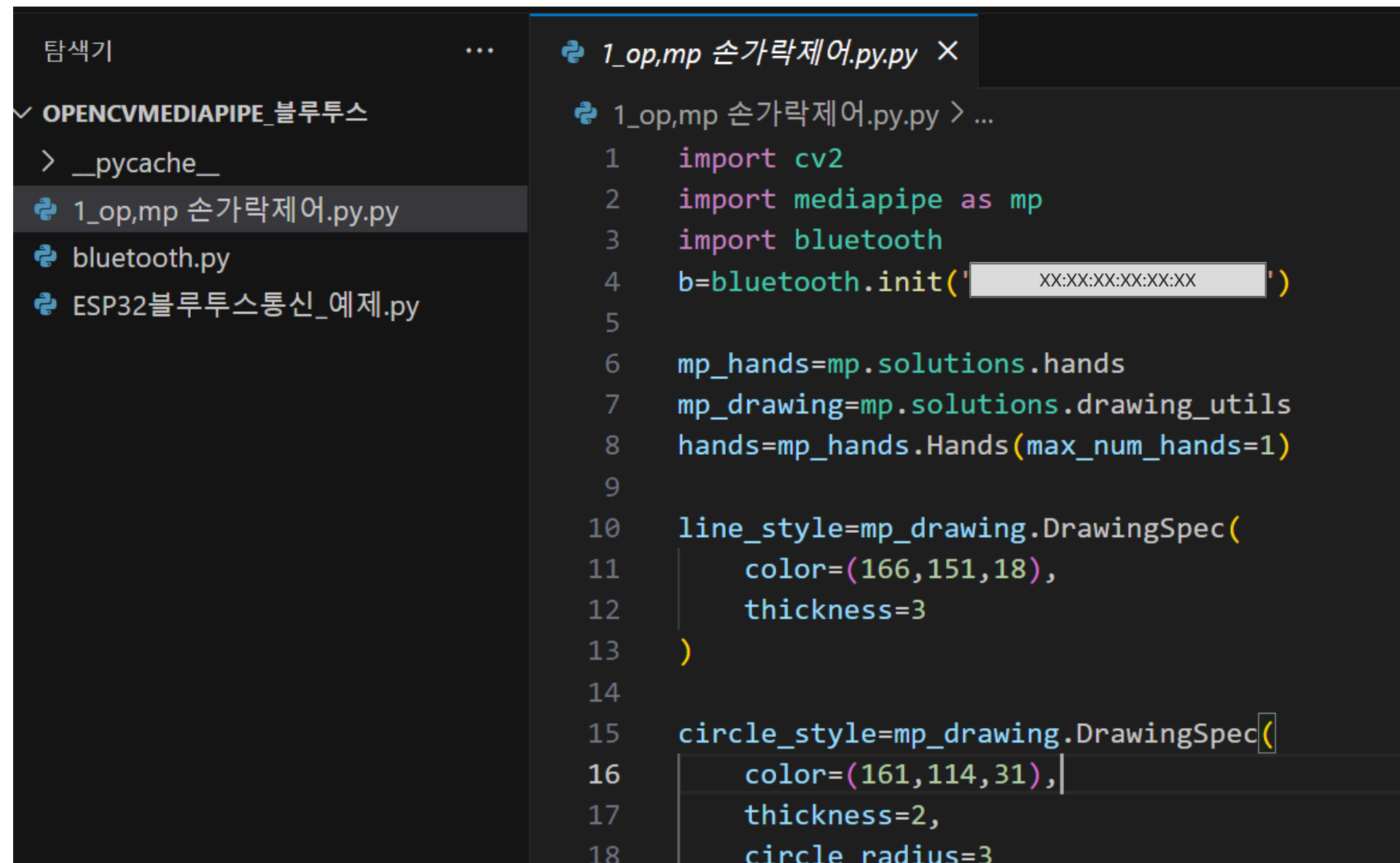
-값을 하나만 보내는 경우

(값을 던짐)-local python

가림 — 학급 게시물 화면

압축을 푼 후 vscode에서 폴더열기로 폴더를 열어준다.

(값을 던짐)-local python



```
1 import cv2
2 import mediapipe as mp
3 import bluetooth
4 b=bluetooth.init('XX:XX:XX:XX:XX:XX')
5
6 mp_hands=mp.solutions.hands
7 mp_drawing=mp.solutions.drawing_utils
8 hands=mp_hands.Hands(max_num_hands=1)
9
10 line_style=mp_drawing.DrawingSpec(
11     color=(166,151,18),
12     thickness=3
13 )
14
15 circle_style=mp_drawing.DrawingSpec(
16     color=(161,114,31),
17     thickness=2,
18     circle_radius=3
```

폴더 열기 한 상황

(값을 던짐)-local python

```
31     if results.multi_hand_landmarks:
32         for hand_landmarks in results.multi_hand_landmarks:
33             finger_point=hand_landmarks.landmark[8]
34             finger_x=int(finger_point.x*width)
35             finger_y=int(finger_point.y*height)
36             if finger_x<100:
37                 b.send("a")
38             elif finger_x>width-100:
39                 b.send("b")
40             else :
41                 b.send("c")
```

X값의 범위에 따라서 a,b,c 값을 보낸다.

(값을 받음) Thonny- ESP32

가림 — 학급 게시물 화면

Thonny_블루투스를 압축을 푼다.

(값을 받음) Finger_rgb.py 파일을 boot.py에 저장-ESP32

Thonny - C:\Users\User\Downloads\thonny_블루투스\finger_rgb.py @ 12:19

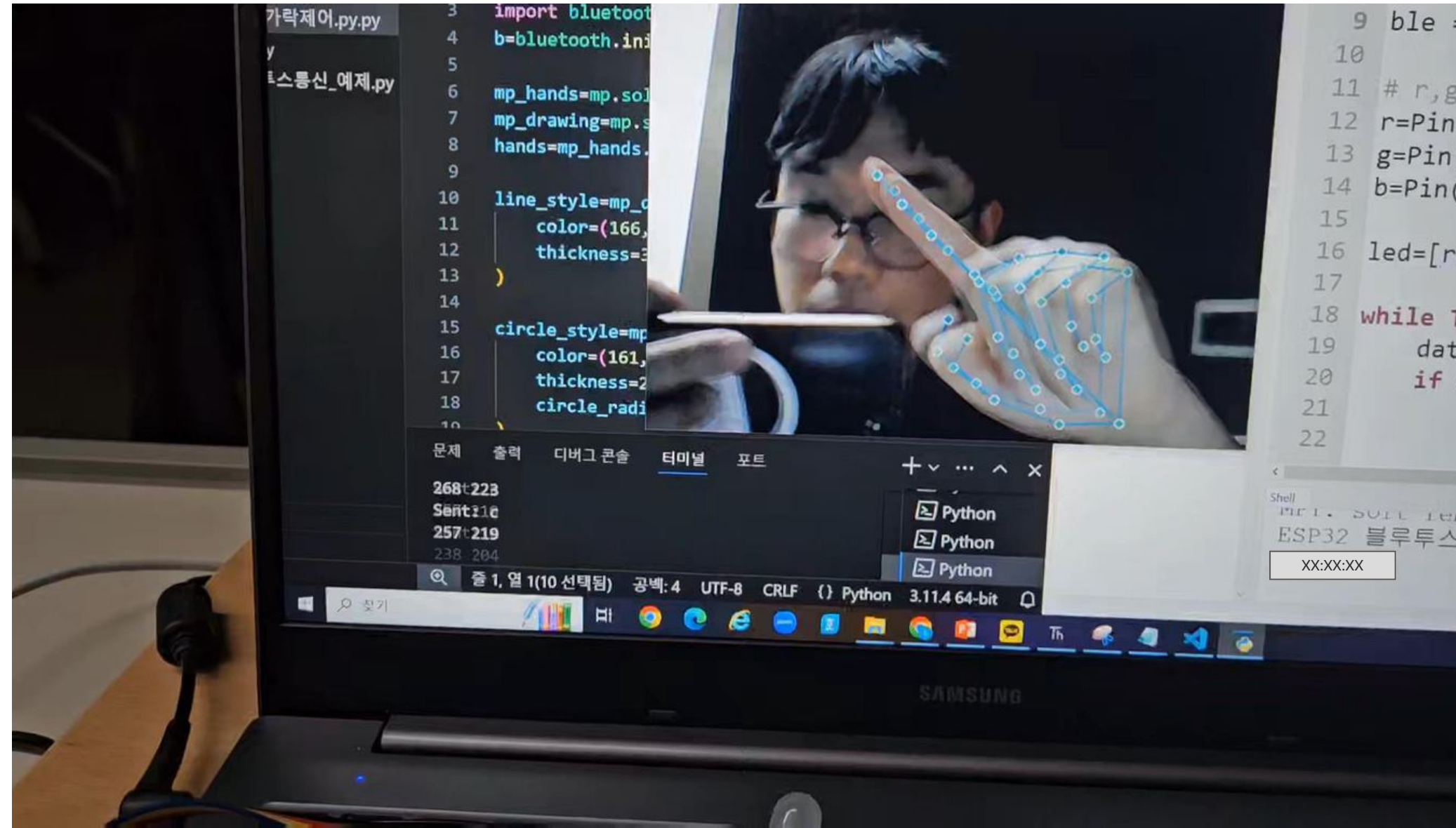
File Edit View Run Tools Help



The screenshot shows the Thonny IDE interface. On the left, the 'Files' pane shows the project structure. The 'This computer' section shows the file explorer with 'Downloads' expanded, showing a folder 'thonny_블루투스' which contains several Python files, including 'finger_rgb.py' which is selected. The 'MicroPython device' section shows files like 'boot.py', 'ESP32BLE.py', 'i2c_lcd.py', and 'mg90s_servo.py'. The main editor window shows the code for 'finger_rgb.py' with line numbers 1 through 15. The code includes comments in Korean and Python code for importing ESP32BLE, time, and Pin, and initializing BLE and pins.

```
1 #ESP32BLE.py 파일을 ESP32에 업로드 후 실행
2 import ESP32BLE
3 import time
4 from machine import Pin
5 from time import sleep
6
7
8 # 블루투스 통신 시작하기
9 ble = ESP32BLE.init("ESP32")
10
11 # r,g,b 값 나타내기
12 r=Pin(27, Pin.OUT)
13 g=Pin(32, Pin.OUT)
14 b=Pin(33, Pin.OUT)
15
```

동작영상



Local python 실행 후 손가락의 움직임에 따른
피지컬 교구의 상태를 살펴본다.

블루투스 통신(응용) 값 주고 받기(손가락으로 RGB제어)

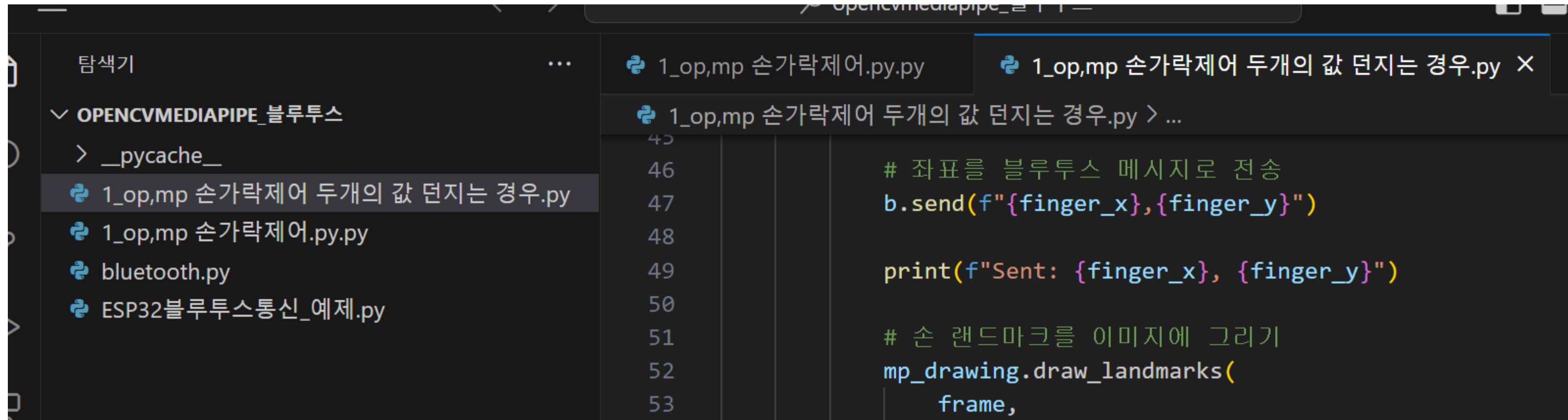
-값을 두개 보내는 경우

(값을 던짐) local python

가림 — 학급 게시물 화면

해당 파일을 읽은 후 thonny에 업로드 해준다.

(값을 던짐) local python



```
45
46 # 좌표를 블루투스 메시지로 전송
47 b.send(f"{finger_x},{finger_y}")
48
49 print(f"Sent: {finger_x}, {finger_y}")
50
51 # 손 랜드마크를 이미지에 그리기
52 mp_drawing.draw_landmarks(
53     frame,
```

업로드된 상황

(값을 던짐) local python

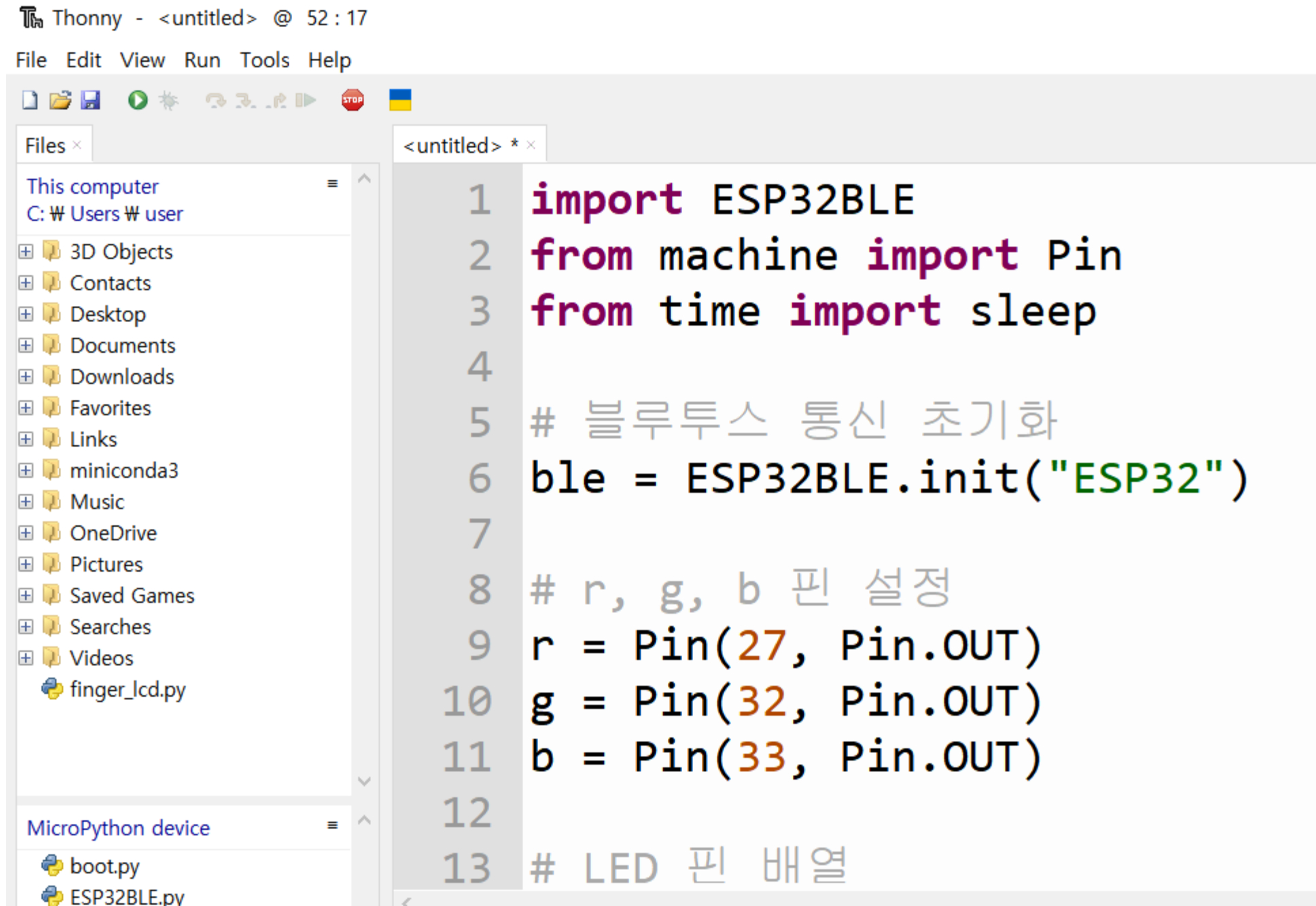
```
42     finger_point = hand_landmarks.landmark[8]
43     finger_x = int(finger_point.x * width)
44     finger_y = int(finger_point.y * height)
45
46     # 좌표를 블루투스 메시지로 전송
47     b.send(f"{finger_x},{finger_y}")
48
49     print(f"Sent: {finger_x}, {finger_y}")
50
51     # 손 랜드마크를 이미지에 그리기
52     mp_drawing.draw_landmarks(
```

(값을 받음) - ESP32- boot.py에 업로드

가림 — 학급 게시물 화면

해당 파일을 읽은 후 Thonny에 업로드를 해준다.

(값을 받음) - ESP32- boot.py에 업로드

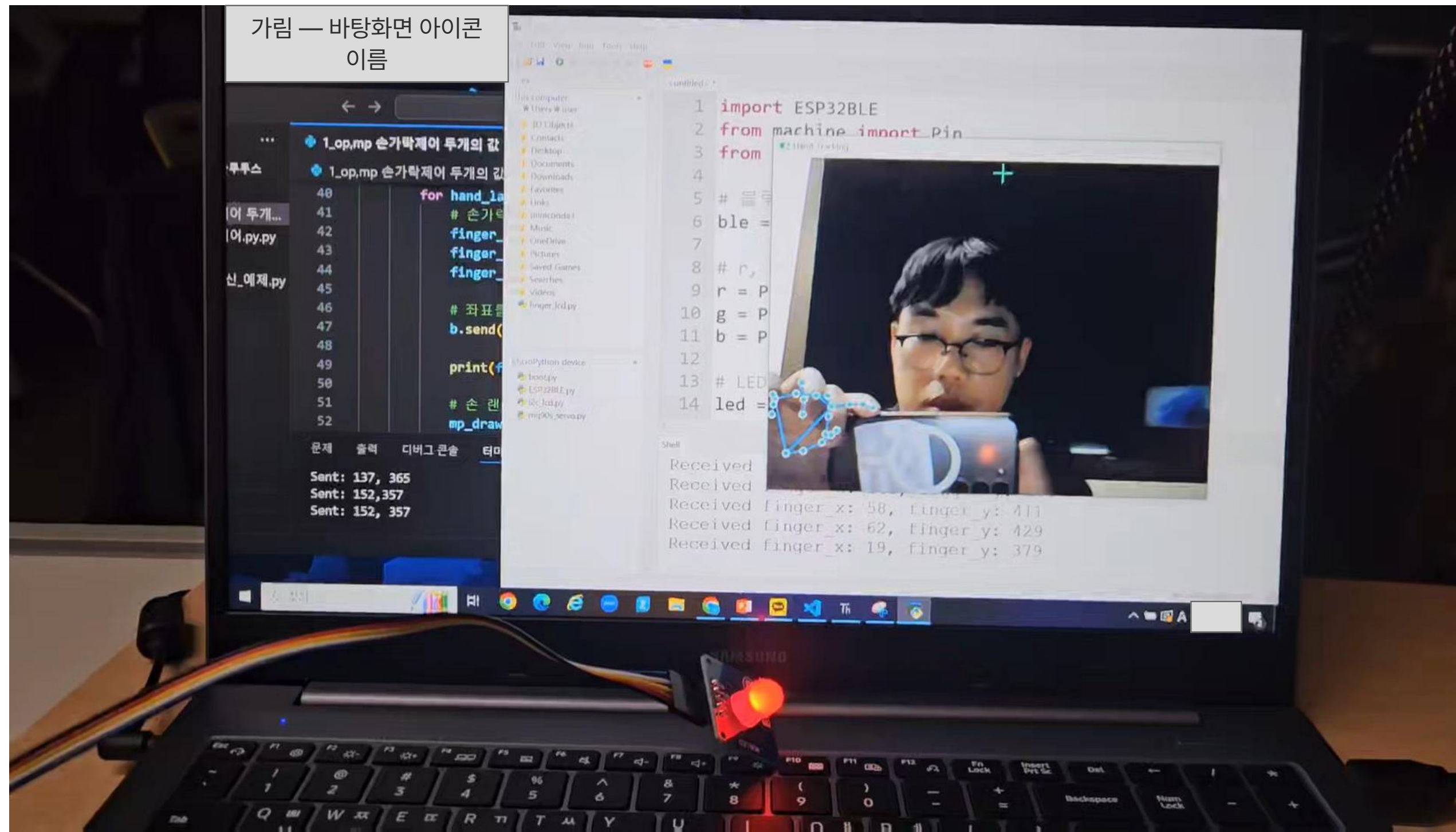


The screenshot shows the Thonny IDE interface. The top bar indicates the file is untitled and the time is 52:17. The menu bar includes File, Edit, View, Run, Tools, and Help. The left sidebar shows the file explorer with 'This computer' and 'MicroPython device' sections. The main editor window displays a Python script for ESP32 BLE communication.

```
1 import ESP32BLE
2 from machine import Pin
3 from time import sleep
4
5 # 블루투스 통신 초기화
6 ble = ESP32BLE.init("ESP32")
7
8 # r, g, b 핀 설정
9 r = Pin(27, Pin.OUT)
10 g = Pin(32, Pin.OUT)
11 b = Pin(33, Pin.OUT)
12
13 # LED 핀 배열
```

Thonny에서 재생 후 vscode에서 업로드 해준다.

동작영상



출처와 라이선스

블루투스 통신 수업 교안(가린 편집본) — 이 쪽은 사이트가 편집본 끝에 덧붙였어요(원본 교안에는 없는 쪽). 1~91쪽은 원본 교안과 쪽 번호가 같아요.

저작자와 라이선스

원본 교안의 저작자는 박상진·김석전이에요. 이 편집본은 CC BY-NC-SA 4.0(저작자표시-비영리-동일조건변경허락 4.0 국제)으로 나뉘요 — 저작자를 밝히고, 돈을 받고 팔지 않고, 고친 것도 같은 조건으로 나누면 누구나 쓰고 고치고 나눌 수 있어요. 라이선스 원문:

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.ko>

원래 권리자의 조건을 따르는 것(위 라이선스에서 빠져요)

20·26·31쪽의 손 랜드마크 번호 그림은 MediaPipe 공식 문서 그림이에요(© The MediaPipe Authors, Apache License 2.0). 4쪽의 OpenCV 로고, 발표 슬라이드 틀의 장식, 화면 캡처에 보이는 프로그램(Thonny·Visual Studio Code 등)의 화면은 각 권리자의 것이에요.

회색 상자

개인정보가 보이던 곳을 지운 자리예요 — 기관 계정 폴더 경로, 개인 파일 이름, 블루투스 기기 주소, 학급 게시물 화면, 외부 홍보 이미지 속 인물. 상자 아래의 글자와 그림까지 지웠고, 파일 속 정보(작성자·만든 프로그램·날짜 기록)도 지웠어요.

예제 코드

교안의 예제 코드는 사이트의 실습실과 예제 갤러리에서 MIT 라이선스로 받을 수 있어요(라이브러리 파일 몇 개는 원래 저작자의 조건을 따라요).

더 알아보기

교사용 자료실 <https://songdocomputerpark-lang.github.io/ai-physical-computing/teacher/>

출처와 라이선스 전체 목록 <https://songdocomputerpark-lang.github.io/ai-physical-computing/credits/>